

XCPC Algorithm Template

零、标准头文件模板

0.0、缺省源

```

1  /*1.深呼吸，不要紧张，慢慢读题，读明白题，题目往往比你想象的简单。2.暴力枚举:枚举什
2  么，是否可以使用一些技巧加快枚举速度（预处理、前缀和、数据结构、数论分块）
3  3.贪心:需要排序或使用数据结构（pq）吗，这么贪心一定最优吗。
4  4.二分：满足单调性吗，怎么二分，如何确定二分函数返回值是什么。5.位运算：按位贪心，还
5  是与位运算本身的性质有关。
6  6.数学题：和最大公因数、质因子、取模是否有关。
7  7.dp: 怎么设计状态，状态转移方程是什么，初态是什么，使用循环还是记忆化转移。8.搜索：
8  dfs 还是 bfs ，搜索的时候状态是什么，需要记忆化吗。
9  9.树上问题：是树形 dp、树上贪心、或者是在树上搜索。
10 10.图论：依靠什么样的关系建图，是求环统计结果还是最短路。
11 11.组合数学：有几种值，每种值如何被组成，容斥关系是什么。
12 12.交互题：log(n)次如何二分，2*n 次如何通过 n 次求出一些值，再根据剩余次数求答案。
13 13.如果以上几种都不是，多半是有一个 point 你没有注意到，记住正难则反。*/
14 #pragma GCC optimize("O3,unroll-loops,Ofast,inline")
15 #include "bits/stdc++.h"
16
17 using i64 = int64_t;
18 using i128 = __int128_t;
19
20 using u32 = uint32_t;
21 using u64 = uint64_t;
22 using u128 = __uint128_t;
23
24 using f32 = float;
25 using f64 = double;
26 using f80 = long double;
27
28 constexpr int INF = 0x3f3f3f3f;
29 constexpr i64 LINF = 0x3f3f3f3f3f3f3f3fLL;
30 constexpr i64 mod = 998'244'353LL;
31 constexpr i64 MOD = 1e9 + 7;
32
33 static inline void VioletEverGraden() {
34
35  /*
36  愿幸运女神与最纯粹的前行者同在。
37  Test Sample
38  */

```

```

36 }
37
38 signed main() {
39 #ifdef Algostruggler
40     freopen("A.in", "r", stdin);
41 #endif // Algostruggler
42
43     std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
44     std::cout.tie(nullptr);
45
46     // std::cout << std::fixed << std::setprecision(12), std::cerr <<
47     std::fixed << std::setprecision(12);
48
49     // for (int _ = [&]() -> int { int T; std::cin >> T; return T; }(); _; _
50     --)
51         VioletEverGraden();
52
53     return 0;
54 }

```

0.1、i128 库函数自定义

```

1 // 输入 i128
2 std::istream &operator >> (std::istream &is, i128 &n) {
3     std::string s;
4     is >> s;
5     n = 0;
6     bool neg = false;
7     size_t i = 0;
8     if (s[0] == '-') {
9         neg = true; i = 1;
10    }
11    for (; i < s.size(); i++) {
12        n = n * 10 + (s[i] - '0');
13    }
14    if (neg) {
15        n = -n;
16    }
17    return is;
18 }
19
20 // 输出 i128
21 std::ostream &operator << (std::ostream &os, i128 n) {
22     if (n == 0) {
23         return os << 0;
24     }
25     std::string s;
26     while (n > 0) {
27         s += char('0' + n % 10);
28         n /= 10;

```

```

29     }
30     std::reverse(s.begin(), s.end());
31     return os << s;
32 }
33
34 // 字符串转 i128
35 i128 toi128(const std::string &s) {
36     i128 n = 0;
37     for (auto c : s) {
38         n = n * 10 + (c - '0');
39     }
40     return n;
41 }
42
43 // 整数平方根（向下取整）
44 i128 sqrti128(i128 n) {
45     i128 lo = 0, hi = 1E16;
46     while (lo < hi) {
47         i128 x = (lo + hi + 1) / 2;
48         if (x * x ≤ n) {
49             lo = x;
50         } else {
51             hi = x - 1;
52         }
53     }
54     return lo;
55 }
56
57 // 最大公约数（非负）
58 i128 gcd(i128 a, i128 b) {
59     while (b) {
60         a %= b;
61         std::swap(a, b);
62     }
63     return a;
64 }

```

0.2、常用库函数重载

```

1 // 上取整下取整
2 i64 ceilDiv(i64 n, i64 m) {
3     if (n ≥ 0) {
4         return (n + m - 1) / m;
5     } else {
6         return n / m;
7     }
8 }
9
10 i64 floorDiv(i64 n, i64 m) {
11     if (n ≥ 0) {

```

```

12     return n / m;
13 } else {
14     return (n - m + 1) / m;
15 }
16 }
17
18 // 最大值赋值
19 template<class T>
20 void chmax(T &a, T b) {
21     if (a < b) {
22         a = b;
23     }
24 }
25
26 // 最大公约数
27 i128 gcd(i128 a, i128 b) {
28     return b ? gcd(b, a % b) : a;
29 }
30
31 // 精确开平方
32 i64 sqrt(i64 n) {
33     i64 s = std::sqrt(n);
34     while (s * s > n) {
35         s--;
36     }
37     while ((s + 1) * (s + 1) ≤ n) {
38         s++;
39     }
40     return s;
41 }
42
43 // 精确开平方
44 i64 get(i64 n) {
45     i64 u = std::sqrt(2.0L * n);
46     while (u * (u + 1) / 2 < n) {
47         u++;
48     }
49     while (u * (u - 1) / 2 + 1 > n) {
50         u--;
51     }
52     return u;
53 }
54
55 // 求 Log
56 int logi(int a, int b) {
57     int t = 0;
58     i64 v = 1;
59     while (v < b) {
60         v *= a;
61         t++;
62     }
63     return t;

```

```

64 }
65
66 int llog(int a, int b) {
67     if (a ≤ b) {
68         int l = logi(a, b);
69         return (l == 0 ? 0 : std::__lg(2 * l - 1));
70     }
71     int l = logi(b, a + 1) - 1;
72     assert(l > 0);
73     return -std::__lg(l);
74 }

```

一、基础算法

1.1、二分与三分

整数二分

- 在递增序列 A 中查找 $\geq x$ 数中最小的一个（即 x 或 x 的后继）。

```

1 int L = 0, R = 1E8, ans = R;
2 while (L ≤ R) {
3     int mid = L + R >> 1;
4     if (check(mid)) {
5         R = mid - 1;
6         ans = mid;
7     } else {
8         L = mid + 1;
9     }
10 }
11 return ans;

```

- 在递增序列 A 中查找 $\leq x$ 数中最大的一个（即 x 或 x 的前驱）。

```

1 int L = 0, R = 1E8, ans = L;
2 while (L ≤ R) {
3     int mid = L + R >> 2;
4     if (check(mid)) {
5         L = mid + 1;
6         ans = mid;
7     } else {
8         R = mid - 1;
9     }
10 }
11 return ans;

```

实数二分

目前主流的写法是限制二分次数。

```

1  using f80 = long double;
2  for (int t = 1; t ≤ 100; t++) {
3      f80 mid = (L + R) / 2;
4      if (check(mid)) {
5          r = mid;
6      } else {
7          L = mid;
8      }
9  }
10 std::cout << L << '\n';

```

整数三分

```

1  while (L < R) {
2      int mid = (L + R) / 2;
3      if (check(mid) ≤ check(mid + 1)) {
4          R = mid;
5      } else {
6          L = mid + 1;
7      }
8  }
9  std::cout << check(L) << '\n';

```

实数三分

```

1  using f80 = long double;
2  f80 L = -1E9, R = 1E9;
3  for (int t = 1; t ≤ 100; t++) {
4      f80 mid1 = (L * 2 + R) / 3;
5      f80 mid2 = (L + R * 2) / 3;
6      if (check(mid1) < check(mid2)) {
7          R = mid2;
8      } else {
9          L = mid1;
10     }
11 }
12 std::cout << L << endl;

```

二、STL 与库函数

2.1 pb_ds 库

其中 gp_hash_table 使用的最多，其等价于 unordered_map，内部是无序的。

```
1 #include <bits/exec.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 template<class S, class T> using omap = __gnu_pbds::gp_hash_table<S, T,
  myhash>;
```

2.2、查找后继 lower_bound、upper_bound

lower 表示 $\geq$ ，upper 表示 $>$ 。使用前记得先进行排序

```
1 //返回a数组[start,end)区间中第一个  $\geq x$  的地址【地址!!!】
2 cout << lower_bound(a + start, a + end, x);
3
4 cout << lower_bound(a, a + n, x) - a; //在a数组中查找第一个  $\geq x$  的元素下标
5 upper_bound(a, a + n, k) - lower_bound(a, a + n, k); //查找k在a中出现了几次
```

2.3、数组打乱 shuffle

```
1 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
2 shuffle(ver.begin(), ver.end(), rnd);
```

2.4、二分搜索 binary_search

用于查找某一元素是否在容器中，相当于 find 函数。在使用前需要先进行排序。

```
1 //在a数组[start,end)区间中查找x是否存在，返回bool型
2 cout << binary_search(a + start, a + end, x);
```

2.5、批量递增赋值函数 iota

```
1 //将a数组[start,end)区间赋值成“x, x+1, x+2, ...”
2 iota(a + start, a + end, x);
```

2.6、数组去重函数 *unique*

在使用前需要先进行排序。

其作用是，对于区间 [开始位置, 结束位置]，不停的把后面不重复的元素移到前面来，也可以说是用 **不重复的元素占领重复元素的位置**。并且返回去重后容器中不重复序列的最后一个元素的下一个元素。所以在进行操作后，数组、容器的大小并没有发生改变。

```
1 //将a数组[start,end)区间去重，返回迭代器
2 unique(a + start, a + end);
3
4 //与erase函数结合，达到去重+删除的目的
5 a.erase(unique(ALL(a)), a.end());
```

2.7、位运算函数 *builtin*

```
1 __builtin_popcount(x) // 返回x二进制下含1的数值，例如x=15=(1111)时答案为4
2 __builtin_ffs(x)      // 返回x右数第一个1的位置(1-idx)，1(1) 返回 1，8(1000) 返回
3                        4，26(11010) 返回 2
4 __builtin_ctz(x)      // 返回x二进制下后导0的个数，1(1) 返回 0，8(1000) 返回 3
5 bit_width(x)          // 返回x二进制下的位数，9(1001) 返回 4，26(11010) 返回 5
```

注：以上函数的 `long long` 版本只需要在函数后面加上 `ll` 即可（例如 `__builtin_popcountll(x)`），`unsigned long long` 加上 `ull`。

2.8、数字转字符串函数

`itoa` 虽然能将整数转换成任意进制的字符串，但是其不是标准的C函数，且为Windows独有，且不支持 `long long`，建议手写。

```
1 // to_string函数会直接将你的各种类型的数字转换为字符串。
2 // string to_string(T val);
3 double val = 12.12;
4 cout << to_string(val);
```

```
1 // 【不建议使用】itoa允许你将整数转换成任意进制的字符串，参数为待转换整数、目标字符串组、进制。
2 // char* itoa(int value, char* string, int radix);
3 char ans[10] = {};
4 itoa(12, ans, 2);
5 cout << ans << endl; /*1100*/
6
7 // 长整型函数名ltoa，最高支持到int型上限2^31。ultoa同理。
```


2.9、字符串转数字

```

1 // stoi直接使用
2 cout << stoi("12") << endl;
3
4 // 【不建议使用】stoi转换进制，参数为待转换字符串、起始位置、进制。
5 // int stoi(string value, int st, int radix);
6 cout << stoi("1010", 0, 2) << endl; /*10*/
7 cout << stoi("c", 0, 16) << endl;   /*12*/
8 cout << stoi("0x3f3f3f3f", 0, 0) << endl; /*1061109567*/
9
10 // 长整型函数名stoll，最高支持到long类型上限 $2^{63}$ 。stoull、stod、stold同理。

```

```

1 // atoi直接使用，空字符返回0，允许正负符号，数字字符前有其他字符返回0，数字字符前有空
  字符自动去除
2 cout << atoi("12") << endl;
3 cout << atoi(" -12") << endl;   /*12*/
4 cout << atoi("-12abc") << endl; /*-12*/
5 cout << atoi("abc12") << endl;  /*0*/
6
7 // 长整型函数名atoll，最高支持到long类型上限 $2^{63}$ 。

```

2.10、全排列算法 *next_permutation*、*prev_permutation*

在提及这个函数时，我们先需要补充几点字典序相关的知识。

对于三个字符所组成的序列 {a,b,c}，其按照字典序的6种排列分别为：

{abc}, {acb}, {bac}, {cba}, {cab}, {cba}

其排序原理是：先固定 a（序列最小元素），再对之后的元素排列。而 $b < c$ ，所以 $abc < acb$ 。

同理，先固定 b（序列次小元素），再对之后的元素排列。即可得出以上序列。

next_permutation 算法，即是按照字典序顺序输出的全排列；相对应的，*prev_permutation* 则是按照逆字典序顺序输出的全排列。可以是数字，亦可以是其他类型元素。其直接在序列上进行更新，故直接输出序列即可。

```

1  int n;
2  cin >> n;
3  vector<int> a(n);
4  // iota(a.begin(), a.end(), 1);
5  for (auto & it : a) cin >> it;
6  sort(a.begin(), a.end());
7
8  do {
9      for (auto it : a) cout << it << " ";
10     cout << endl;
11 } while (next_permutation(a.begin(), a.end()));

```

2.11、字符串转换为数值函数 *stoi*

可以快捷的将一串字符串转换为指定进制的数字。

使用方法

- `stoi` (字符串, 0, x进制): 将一串 x 进制的字符串转换为 int 型数字。

```

1  void Solve() {
2      cout << stoi("1010", 0, 2) << endl;
3      cout << stoi("c", 0, 16) << endl;
4      cout << stoi("0x3f3f3f3f", 0, 0) << endl;
5      cout << stoi("10", 0, 8) << endl;
6      cout << stoll("aaaaaaaa", 0, 16) << endl;
7  }

```

输出示例:

```

1  10
2  12
3  1061109567
4  8
5  11728124029610

```

2.12、数值转换为字符串函数 *to_string*

允许将各种数值类型转换为字符串类型。

```

1  // 将数值num转换为字符串
2  string s = to_string(num);

```

2.13、判断非递减 *is_sorted*

```
1 // a数组[start,end)区间是否是非递减的，返回bool型
2 cout << is_sorted(a + start, a + end);
```

2.14、累加 *accumulate*

```
1 // 将a数组[start,end)区间的元素进行累加，并输出累加和+x的值
2 cout << accumulate(a + start, a + end, x);
```

2.15、迭代器 *iterator*

```
1 // 构建一个Uuu容器的正向迭代器，名字叫it
2 Uuu::iterator it;
3
4 vector<int>::iterator it; // 创建一个正向迭代器，++ 操作时指向下一个
5 vector<int>::reverse_iterator it; // 创建一个反向迭代器，++ 操作时指向上一个
```

2.16、其他函数

`exp2(x)` : 返回 2^x

`log2(x)` : 返回 $\log_2(x)$

`gcd(x, y) / lcm(x, y)` : 以 $\log$ 的复杂度返回 $\gcd(|x|, |y|)$ 与 $\text{lcm}(|x|, |y|)$ ，且返回值符号也为正数。

2.17、容器与成员函数

元组 *tuple*

```
1 // 获取obj对象中的第index个元素-get<index>(obj)
2 // 需要注意的是这里的index只能手动输入，使用for循环这样的自动输入是不可以的
3 tuple<string, int, int> Student = {"Wida", 23, 45000};
4 cout << get<0>(Student) << endl; // 获取Student对象中的第一个元素，这里的输出结果
   应为"Wida"
```

数组 array

```
1 array<int, 3> x; // 建立一个包含三个元素的数组x
2
3 // 跟正常数组一样，可以使用随机访问
4 cout << x[0]; // 获取数组的第一个元素
```

变长数组 vector

```
1 resize(n)          // 重设容器大小，但是不改变已有元素的值
2 assign(n, 0)       // 重设容器大小为 n，且替换容器内的内容为 0
3
4 // 尽量不要使用[]的形式声明多维变长数组，而是使用嵌套的方式替代
5 vector<int> ver[n + 1];           // 不好的声明方式
6 vector<vector<int>> ver(n + 1);
7
8 // 嵌套时只需要在最后一个注明变量类型
9 vector dis(n + 1, vector<int>(m + 1));
10 vector dis(m + 1, vector(n + 1, vector<int>(n + 1)));
```

栈 stack

栈顶入，栈顶出。先进后出。

```
1 // 没有clear函数
2 size() / empty()
3 push(x)    // 向栈顶插入 x
4 top()      // 获取栈顶元素
5 pop()      // 弹出栈顶元素
```

队列 queue

队尾进，队头出。先进先出。

```
1 // 没有clear函数
2 size() / empty()
3 push(x)    // 向队尾插入 x
4 front() / back() // 获取队头、队尾元素
5 pop()      // 弹出队头元素
6
7 // 没有clear函数，但是可以用重新构造替代
8 queue<int> q;
9 q = queue<int>();
```

双向队列 deque

```

1 size() / empty() / clear()
2 push_front(x) / push_back(x)
3 pop_front(x) / pop_back(x)
4 front() / back()
5 begin() / end()
6 []

```

优先队列 priority_queue

默认升序（大根堆），自定义排序需要重载 `<`。

```

1 // 没有clear函数
2 priority_queue<int, vector<int>, greater<int>> p; // 重定义为降序（小根堆）
3 push(x); // 向堆中插入 x
4 top(); // 获取堆顶元素
5 pop(); // 弹出堆顶元素
6 // 重载运算符【注意，符号相反!!!】
7 struct Node {
8     int x; string s;
9     friend bool operator < (const Node &a, const Node &b) {
10         if (a.x != b.x) return a.x > b.x;
11         return a.s > b.s;
12     }
13 };

```

字符串 string

```

1 size() / empty() / clear()
2
3 // 从字符串s的S[start]开始，取出长度为len的子串-S.substr(start, len)
4 // len省略时默认取到结尾，超过字符串长度时也默认取到结尾
5 cout << S.substr(1, 12);
6
7 find(x) / rfind(x); // 顺序、逆序查找x，返回下标，没找到时返回一个极大值【! 建议
// 与 size() 比较，而不要和 -1 比较，后者可能出错】
8 // 注意，没有count函数

```

有序、多重有序集合 set、multiset

默认升序（大根堆），set 去重，multiset 不去重， $O(\log N)$ 。

```

1  set<int, greater<> > s;    // 重定义为降序（小根堆）
2  size() / empty() / clear()
3  begin() / end()
4  ++ / --    // 返回前驱、后继
5
6  insert(x);           // 插入x
7  find(x) / rfind(x);  // 顺序、逆序查找x，返回迭代器【迭代器!!!】，没找到
                        // 时返回end()
8  count(x);            // 返回x的个数
9  lower_count(x);      // 返回第一个 ≥ x的迭代器【迭代器!!!】
10 upper_count(x);      // 返回第一个 > x的迭代器【迭代器!!!】

```

特殊函数 next 和 prev 详解：

```

1  auto it = s.find(x);      // 建立一个迭代器
2  prev(it) / next(it);     // 默认返回迭代器it的前/后一个迭代器
3  prev(it, 2) / next(it, 2); // 可选参数可以控制返回前/后任意个迭代器
4
5  /* 以下是一些应用 */
6  auto pre = prev(s.lower_bound(x)); // 返回第一个 < x的迭代器
7  int ed = *prev(S.end(), 1);        // 返回最后一个元素

```

erase(x); 有两种删除方式：

- 当 `x` 为某一元素时，删除所有这个数，复杂度为 $O(num_x + \log N)$;
- 当 `x` 为迭代器时，删除这个迭代器。

```

1  // 连续头部删除
2  set<int> S = {0, 9, 98, 1087, 894, 34, 756};
3  auto it = S.begin();
4  int len = S.size();
5  for (int i = 0; i < len; ++i) {
6      if (*it ≥ 500) continue;
7      it = S.erase(it); // 删除所有小于500的元素
8  }
9  // 错误用法如下【千万不能这样用!!!】
10 // for (auto it : S) {
11 //     if (it ≥ 500) continue;
12 //     S.erase(it); // 删除所有小于500的元素
13 // }

```

map、multimap

默认升序（大根堆），map 去重，multimap 不去重， $O(\log S)$ ，其中 S 为元素数量。

```
1 map<int, int, greater<> > mp;    // 重定义为降序（小根堆）
2 size() / empty() / clear()
3 begin() / end()
4 ++ / --    // 返回前驱、后继
5
6 insert({x, y});    // 插入二元组
7 []            // 随机访问，multimap不支持
8 count(x);        // 返回x为下标的个数
9 lower_bound(x);   // 返回第一个下标 ≥ x的迭代器
10 upper_bound(x);  // 返回第一个下标 > x的迭代器
```

erase(x); 有两种删除方式：

- 当 x 为某一元素时，删除所有以这个元素为下标的二元组，复杂度为 $O(num_x + \log N)$;
- 当 x 为迭代器时，删除这个迭代器。

慎用随机访问！ ——当不确定某次查询是否存在于容器中时，不要直接使用下标查询，而是先使用 count() 或者 find() 方法检查key值，防止不必要的零值二元组被构造。

```
1 int q = 0;
2 if (mp.count(i)) q = mp[i];
```

慎用自带的 pair、tuple 作为key值类型！使用自定义结构体！

```
1 struct fff {
2     LL x, y;
3     friend bool operator < (const fff &a, const fff &b) {
4         if (a.x != b.x) return a.x < b.x;
5         return a.y < b.y;
6     }
7 };
8 map<fff, int> mp;
```

bitset

将数据转换为二进制，从高位到低位排序，以 0 为最低位。当位数相同时支持全部的位运算。

```
1 // 如果输入的是01字符串，可以直接使用">>"读入
2 bitset<10> s;
```

```

3  cin >> s;
4
5  // 使用只含01的字符串构造bitset<容器长度>B (字符串)
6  string S; cin >> S;
7  bitset<32> B (S);
8
9  // 使用整数构造 (两种方式)
10 int x; cin >> x;
11 bitset<32> B1 (x);
12 bitset<32> B2 = x;
13
14 // 构造时，头括号里的数字不能是变量
15 int x; cin >> x;
16 bitset<x> ans; // 错误构造
17
18 [] // 随机访问
19 set(x)      // 将第x位置1, x省略时默认全部位置1
20 reset(x)    // 将第x位置0, x省略时默认全部位置0
21 flip(x)     // 将第x位取反, x省略时默认全部位取反
22 to_ulong()  // 重转换为ULL类型
23 to_string() // 重转换为ULL类型
24 count()     // 返回1的个数
25 any()       // 判断是否至少有一个1
26 none()      // 判断是否全为0
27
28 bitset<23> B1("11101001"), B2("11101000");
29 cout << (B1 ^ B2) << "\n"; // 按位异或
30 cout << (B1 | B2) << "\n"; // 按位或
31 cout << (B1 & B2) << "\n"; // 按位与
32 cout << (B1 == B2) << "\n"; // 比较是否相等
33 cout << B1 << " " << B2 << "\n"; // 你可以直接使用cout输出

```

2.18、程序标准化

使用 Lambda 函数

- **function 统一写法**

需要注意的是，虽然 `function` 定义时已经声明了返回值类型了，但是有的时候会出错（例如，声明返回 `long long` 但是返回 `int`，原因没去了解），所以推荐在后面使用 `->` 再行声明一遍。


```

1 function<void(int, int)> clac = [&](int x, int y) → void {
2     ;
3     clac(1, 2);
4 }
5 function<bool(int)> dfs = [&](int x) → bool {
6     return dfs(x + 1);
7 };
8 dfs(1);

```

- **auto** 非递归写法

不需要使用递归函数时，直接用 `auto` 替换 `function` 即可。

```

1 auto clac = [&](int x, int y) → void {
2     ;
3 }

```

- **auto** 递归写法

相较于 `function` 写法，需要额外引用一遍自身。

```

1 auto dfs = [&](auto self, int x) → bool {
2     return self(self, x + 1);
3 };
4 dfs(dfs, 1);

```

使用构造函数

可以将一些必要的声明和预处理放在构造函数，在编译时，无论放置在程序的哪个位置，都会先于主函数进行。下方是我将输入流控制声明的过程。

```

1 int __FAST_IO__ = []() { // 函数名称可以随意修改
2     ios::sync_with_stdio(0), cin.tie(0);
3     cout.tie(0);
4     cout << fixed << setprecision(12);
5     freopen("out.txt", "r", stdin);
6     freopen("in.txt", "w", stdout);
7     return 0;
8 };

```

三、图论

3.1、树的直径

树上任意两节点之间最长的简单路径即为树的「直径」。

显然，一棵树可以有多条直径，他们的长度相等。

两次 *dfs*

定理：在一棵树上，从任意节点 y 开始进行一次 DFS，到达的距离其最远的节点 z 必为直径的一端。

```

1  #include <bits/stdc++.h>
2
3  void VioletEverGraden() {
4      int n;
5      std::cin >> n;
6
7      int tot = 1;
8      struct Edge {
9          int to; int nxt; int w;
10     };
11     std::vector<int> head(n + 1, -1);
12     std::vector<Edge> G(2 * n + 10);
13     auto AddEdge = [&](int u, int v, int w = 0) -> void {
14         G[tot].to = v;
15         G[tot].nxt = head[u];
16         G[tot].w = w;
17         head[u] = tot ++;
18     };
19
20     int __ = n - 1;
21     while (__ --) {
22         int u, v;
23         std::cin >> u >> v;
24         AddEdge(u, v); AddEdge(v, u);
25     }
26
27     std::vector<int> dis(n + 1, 0); int c;
28     auto dfs = [&](auto&& dfs, int u, int fa) -> void {
29         for (int i = head[u]; ~i; i = G[i].nxt) {
30             int v = G[i].to;
31             if (v == fa) {
32                 continue;
33             }
34             dis[v] = dis[u] + 1;
35             if (dis[v] > dis[c]) {
36                 c = v;

```

```

37     }
38     dfs(dfs, v, u);
39 }
40 };
41
42     dfs(dfs, 1, 0);
43     dis[c] = 0;
44     dfs(dfs, c, 0);
45
46     std::cout << dis[c] << '\n';
47 }
48
49 signed main() {
50     std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
51     std::cout.tie(nullptr);
52     // std::cout << std::fixed << std::setprecision(12), std::cerr <<
53     std::fixed << std::setprecision(12);
54     // for (int _ = [&]() -> int { int T; std::cin >> T; return T; }(); _; _
55     --)
56         VioletEverGraden();
57     return 0;
58 }

```

树形 DP

树形 DP 可以在存在负权边的情况下求解出树的直径

```

1  #include <bits/stdc++.h>
2
3  void VioletEverGraden() {
4      int n;
5      std::cin >> n;
6
7      int tot = 1;
8      struct Edge {
9          int to; int nxt; int w;
10     };
11     std::vector<int> head(n + 1, -1);
12     std::vector<Edge> G(2 * n + 10);
13     auto AddEdge = [&](int u, int v, int w = 0) -> void {
14         G[tot].to = v;
15         G[tot].nxt = head[u];
16         G[tot].w = w;
17         head[u] = tot ++;
18     };
19
20     int __ = n - 1;

```

```

21     while (__ --) {
22         int u, v;
23         std::cin >> u >> v;
24         AddEdge(u, v); AddEdge(v, u);
25     }
26
27     std::vector<int> dp(n + 1, 0);
28     int d = 0;
29     auto dfs = [&](auto &dfs, int u, int fa) -> void {
30         for (int i = head[u]; ~i; i = G[i].nxt) {
31             int v = G[i].to;
32             if (v == fa) {
33                 continue;
34             }
35             dfs(dfs, v, u);
36             d = std::max(d, dp[u] + dp[v] + 1);
37             dp[u] = std::max(dp[u], dp[v] + 1);
38         }
39     };
40
41     dfs(dfs, 1, 0);
42
43     std::cout << d << '\n';
44 }
45
46 signed main() {
47     std::cin.tie(nullptr) ->
48     std::ios_base::sync_with_stdio(false);
49     std::cout.tie(nullptr);
50
51     signed main() {
52         std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
53         std::cout.tie(nullptr);
54
55         // std::cout << std::fixed << std::setprecision(12), std::cerr <<
56         // std::fixed << std::setprecision(12);
57
58         // for (int _ = [&]() -> int { int T; std::cin >> T; return T; }(); _; _
59         --)
60             VioletEverGraden();
61
62         return 0;
63     }

```

3.2、树的中心

在树中，如果节点 x 作为根节点时，从 x 出发的最长链最短，那么称 x 为这棵树的中心。

- 树的中心不一定唯一，但最多有 2 个，且这两个中心是相邻的。

- 树的中心一定位于树的直径上。
- 树上所有点到其最远点的路径一定交会于树的中心。
- 当树的中心为根节点时，其到达直径端点的两条链分别为最长链和次长链。
- 当通过在两棵树间连一条边以合并为一棵树时，连接两棵树的中心可以使新树的直径最小。
- 树的中心到其他任意节点的距离不超过树直径的一半。

```

1  #include <bits/stdc++.h>
2
3  void VioletEverGraden() {
4      int n, m;
5      std::cin >> n >> m;
6
7      int tot = 1;
8      struct Edge {
9          int to; int nxt; int w;
10     };
11     std::vector<int> head(n + 1, -1);
12     std::vector<Edge> G(2 * m + 10);
13     auto AddEdge = [&](int u, int v, int w = 0) -> void {
14         G[tot].to = v;
15         G[tot].nxt = head[u];
16         G[tot].w = w;
17         head[u] = tot ++;
18     };
19
20     int __ = m;
21     while (__ --) {
22         int u, v, w;
23         std::cin >> u >> v >> w;
24         AddEdge(u, v, w); AddEdge(v, u, w);
25     }
26
27     std::vector<int> d1(n + 1, 0), d2(n + 1, 0), up(n + 1, 0);
28     int c1 = 0, c2 = 0, min = 1e9;
29
30     auto dfs1 = [&](auto &dfs1, int u, int fa) -> void {
31         for (int i = head[u]; ~i; i = G[i].nxt) {
32             int v = G[i].to; int w = G[i].w;
33             if (v == fa) {
34                 continue;
35             }
36             dfs1(dfs1, v, u);
37             if (d1[v] + w > d1[u]) {
38                 d2[u] = d1[u];
39                 d1[u] = d1[v] + w;
40             } else if (d1[v] + w > d2[u]) {
41                 d2[u] = d1[v] + w;

```

```

42     }
43 }
44 };
45
46 auto dfs2 = [&](auto &&dfs2, int u, int fa) → void {
47     for (int i = head[u]; ~i; i = G[i].nxt) {
48         int v = G[i].to; int w = G[i].w;
49         if (v == fa) {
50             continue;
51         }
52         up[v] = up[u] + w;
53         if (d1[v] + w != d1[u]) {
54             up[v] = std::max(up[v], d1[u] + w);
55         } else {
56             up[v] = std::max(up[v], d2[u] + w);
57         }
58         dfs2(dfs2, v, u);
59     }
60 };
61
62 auto center = [&]() → void {
63     dfs1(dfs1, 1, 0); dfs2(dfs2, 1, 0);
64     for (int i = 1; i ≤ n; i++) {
65         if (std::max(d1[i], up[i]) < min) {
66             min = std::max(d1[i], up[i]);
67             c1 = i; c2 = 0;
68         } else if (std::max(d1[i], up[i]) == min) {
69             c2 = i;
70         }
71     }
72 };
73
74 center();
75
76 std::cout << c1 << ' ' << c2 << '\n';
77
78 /*
79 Test Sample
80
81 6 5
82 1 2 1
83 1 3 1
84 2 4 1
85 2 5 1
86 3 6 1
87
88 1 0
89
90
91 */
92 }
93

```

```

94 signed main() {
95     std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
96     std::cout.tie(nullptr);
97     // std::cout << std::fixed << std::setprecision(12), std::cerr <<
98     std::fixed << std::setprecision(12);
99     // for (int _ = [&]() → int { int T; std::cin >> T; return T; }(); _;
100     _ --)
101         VioletEverGraden();
102     return 0;
103 }

```

3.3、树的重心

3.4、最近公共祖先

1. 单个节点的 LCA 为其自身: $LCA(\{u\}) = u$
2. 祖先判定: u 是 v 的祖先 **当且仅当** $LCA(u, v) = u$
3. 若 u 不是 v 的祖先, 且 v 也不是 u 的祖先, 则 u, v 分别位于 $LCA(u, v)$ 的两棵不同子树中。
4. 遍历顺序特性:
 - 在前序遍历中, $LCA(S)$ 出现在集合 S 中所有元素**之前**。
 - 在后序遍历中, $LCA(S)$ 出现在集合 S 中所有元素**之后**。
5. 集合合并的 LCA 满足结合律: $LCA(A \cup B) = LCA(LCA(A), LCA(B))$
6. 路径性质: 两点的 LCA 必定位于这两点之间的最短路径上。
7. 距离公式: 设 $h(x)$ 表示节点 x 到树根的距离 (深度), 则树上两点 u, v 之间的距离为: $d(u, v) = h(u) + h(v) - 2 \cdot h(LCA(u, v))$

倍增法

```

1  #include <bits/stdc++.h>
2
3  using i64 = long long;
4
5  void VioletEverGraden() {
6      int n, m, r;
7      std::cin >> n >> m >> r;
8
9      int tot = 1;
10     std::vector<int> head(n + 1, -1);
11     struct Edge {

```

```

12     int to; int nxt; int w;
13 };
14 std::vector<Edge> G(2 * m + 10);
15 auto AddEdge = [&](int u, int v, int w = 0) -> void {
16     G[tot].to = v;
17     G[tot].nxt = head[u];
18     G[tot].w = w;
19     head[u] = tot ++;
20 };
21
22 int __ = n - 1;
23 while (__ --) {
24     int u, v;
25     std::cin >> u >> v;
26     AddEdge(u, v); AddEdge(v, u);
27 }
28
29 std::vector<std::vector<int>> fa(31, std::vector<int> (n + 1, 0));
30 std::vector<int> dep(n + 1, 0);
31 auto dfs = [&](auto &dfs, int u, int p) -> void {
32     fa[0][u] = p;
33     dep[u] = dep[p] + 1;
34
35     for (int i = 1; i <= 30; i++) {
36         fa[i][u] = fa[i - 1][fa[i - 1][u]];
37     }
38
39     for (int i = head[u]; ~i; i = G[i].nxt) {
40         int v = G[i].to;
41         if (v == p) {
42             continue;
43         }
44         dfs(dfs, v, u);
45     }
46 };
47
48 dfs(dfs, r, 0);
49
50 auto LCA = [&](int u, int v) -> int {
51     if (dep[u] > dep[v]) {
52         std::swap(u, v);
53     }
54     int dif = dep[v] - dep[u];
55     for (int i = 0; dif; dif >>= 1, i++) {
56         if (dif & 1) {
57             v = fa[i][v];
58         }
59     }
60
61     if (u == v) {
62         return u;
63     }

```



```

64
65     for (int i = 30; i ≥ 0; i --) {
66         if (fa[i][u] ≠ fa[i][v]) {
67             u = fa[i][u];
68             v = fa[i][v];
69         }
70     }
71
72     return fa[0][u];
73 };
74
75 __ = m;
76 while (__ --) {
77     int u, v;
78     std::cin >> u >> v;
79
80     std::cout << LCA(u, v) << '\n';
81 }
82
83 /*
84 Test Sample
85
86 5 5 4
87 3 1
88 2 4
89 5 1
90 1 4
91 2 4
92 3 2
93 3 5
94 1 2
95 4 5
96
97 4
98 4
99 1
100 4
101 4
102
103
104 */
105 }
106
107 signed main() {
108     std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
109     std::cout.tie(nullptr);
110
111     // std::cout << std::fixed << std::setprecision(12), std::cerr <<
112     std::fixed << std::setprecision(12);
113
114     // for (int _ = [&]() → int { int T; std::cin >> T; return T; }(); _;
115     _ --)

```

```

113         VioletEverGraden();
114
115         return 0;
116     }

```

3.5、图的定义

- oriented graph: 有向图
- bidirectional edges: 双向边
- 二分图: 一个图能被分为左右两部分, 任何一条边的两个端点都不在同一部分中。
- 平面图: 若能将无向图 $G = (V, E)$ 画在平面上使得任意两条无重合顶点的边不相交, 则称 G 是平面图。
- 匹配: 一个边的集合, 这些边没有公共顶点。
- 割点 (割顶): 将与某点 i 连接的所有边删去后, 原图分成两个以上不相连的子图, 称 i 为图的割点。
- 点双连通: 在一张连通的无向图中, 对于两个点 u 和 v , 删去任何一个点 (只能删去一个, 且不能删 u 和 v 自己) 它们依旧连通, 则称 u 和 v 点双连通。如果一个图不存在割点, 那么它是一个点双连通图。
- 割边 (桥): 将某边 e 删去后, 原图分成两个以上不相连的子图, 称 e 为图的割边。
- 边双连通: 在一张连通的无向图中, 对于两个点 u 和 v , 删去任何一条边 (只能删去一条) 它们依旧连通, 则称 u 和 v 边双连通。一个图如果不存在割边, 则它是一个边双连通图。
- 无向正权图上某一点的偏心距: 记为 $ecc(u) = \max dist(u, v)$ 。
- 图的直径: 定义为 $d = \max ecc(u)$ 。
- 图的中心: 定义为 $arg = \min ecc(u)$ 。
- 图的绝对中心: 可以定义在边上的图的中心。
- 图的半径: 定义为 $r = \min ecc(u)$ 。

3.6、拓扑排序

假设这个图 $G = (V, E)$ 在初始化入度为 0 的集合 S 的时候就需要遍历整个图, 并检查每一条边, 因而有 $O(E + V)$ 的复杂度。然后对该集合进行操作, 显然也是需要 $O(E + V)$ 的时间复杂度。

因而总的时间复杂度为 $O(E + V)$ 。

```

1 | #include <bits/stdc++.h>
2 |

```

```

3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5  const int MAXN = 0;
6
7  int n, m;
8  int u, v;
9
10 void violet() {
11     std::cin >> n >> m;
12
13     int tot = 1;
14     std::vector<int> head(n + 1, -1);
15     struct Edge {
16         int to; int nxt;
17     };
18     std::vector<Edge> G(m + 10);
19     auto AddEdge = [&](int u, int v) → void {
20         G[tot].to = v;
21         G[tot].nxt = head[u];
22         head[u] = tot ++;
23     };
24
25     int __ = m;
26     std::vector<int> indeg(n + 1, 0);
27     while (__ --) {
28         std::cin >> u >> v;
29         AddEdge(u, v);
30         indeg[v] ++;
31     }
32
33     std::vector<int> topo; topo.push_back(-1);
34     auto toposort = [&]() → void {
35         std::queue<int> q;
36         rep(i, 1, n, 1) {
37             if (indeg[i] == 0) {
38                 q.push(i);
39             }
40         }
41
42         while (!q.empty()) {
43             auto u = q.front();
44             q.pop();
45             topo.push_back(u);
46
47             for (int i = head[u]; ~i; i = G[i].nxt) {
48                 int v = G[i].to;
49                 indeg[v] --;
50                 if (indeg[v] == 0) {
51                     q.push(v);
52                 }
53             }
54         }

```

```

55     };
56
57     toposort();
58
59     if (topo.size() == n + 1) {
60         for (int i = 1; i ≤ n; i++) {
61             std::cout << topo[i] << " \n"[i == n];
62         }
63     } else {
64         std::cout << -1 << '\n';
65     }
66
67     /*
68     Test Sample
69     5 4
70     1 2
71     2 3
72     3 4
73     4 5
74
75     1 2 3 4 5
76     */
77 }
78
79 signed main() {
80     std::cin.tie(nullptr)→sync_with_stdio(false);
81     std::cout.tie(nullptr);
82
83     int _ = 1;
84     // for (std::cin >> _; _; _--)
85         violet();
86
87     return 0;
88 }

```

合理性证明

考虑一个图，删掉某个入度为 0 的节点之后，如果新图可以拓扑排序，那么原图一定也可以。反过来，如果原图可以拓扑排序，那么删掉后也可以。

应用

拓扑排序可以判断图中是否有环，还可以用来判断图是否是一条链。拓扑排序可以用来求 AOE 网中的关键路径，估算工程完成的最短时间。

求字典序最大/最小的拓扑排序

将 *Kahn* 算法中的队列替换成最大堆/最小堆实现的优先队列即可，此时总的时间复杂度为 $O(E + V \log V)$ 。

3.7、最短路

性质

对于边权为正的图，任意两个结点之间的最短路，不会经过重复的结点。

对于边权为正的图，任意两个结点之间的最短路，不会经过重复的边。

对于边权为正的图，任意两个结点之间的最短路，任意一条的结点数不会超过 (n) ，边数不会超过 $(n-1)$ 。

Floyd

是全源最短路。

适用于任何图，不管有向无向，边权正负，但是最短路必须存在。（不能有负环）

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  const int INF = 0x3f3f3f3f;
5  const int MAXN = 0;
6
7  void violet() {
8      int n;
9      std::cin >> n;
10
11     std::vector<std::vector<int>> G(n + 1, std::vector<int> (n + 1, INF));
12     for (int i = 1; i ≤ n; i++) {
13         for (int j = 1; j ≤ n; j++) {
14             std::cin >> G[i][j];
15             if (G[i][j] == -1) {
16                 G[i][j] = INF;
17             }
18         }
19     }
20
21     for (int k = 1; k ≤ n; k++) {
22         for (int u = 1; u ≤ n; u++) {
23             for (int v = 1; v ≤ n; v++) {
24                 G[u][v] = std::min(G[u][v], G[u][k] + G[k][v]);
25             }
26         }
27     }
28
29     for (int u = 1; u ≤ n; u++) {
30         for (int v = 1; v ≤ n; v++) {
31             if (G[u][v] == INF) {
32                 std::cout << -1 << " \n"[v == n];
33             } else {

```

```

34         std::cout << G[u][v] << " \n"[v == n];
35     }
36 }
37 }
38
39 /*
40 Test Sample
41
42 4
43 0 1 -1 -1
44 -1 0 1 -1
45 -1 -1 0 1
46 1 -1 -1 0
47
48 0 1 2 3
49 3 0 1 2
50 2 3 0 1
51 1 2 3 0
52 */
53 }
54
55 signed main() {
56     std::cin.tie(nullptr) -> sync_with_stdio(false);
57     std::cout.tie(nullptr);
58
59     int _ = 1;
60     // for (std::cin >> _; _; _ --)
61         violet();
62
63     return 0;
64 }

```

Bellman-Ford

Bellman-Ford 算法是一种基于松弛 (*relax*) 操作的最短路算法，可以求出有负权的图的最短路，并可以对最短路不存在的情况进行判断。

对于边 (u, v) ，松弛操作对应下面的式子：

$$\text{dis}(v) = \min(\text{dis}(v), \text{dis}(u) + w(u, v)).$$

Bellman-Ford 算法所做的，就是不断尝试对图上每一条边进行松弛。我们每进行一轮循环，就对图上所有的边都尝试进行一次松弛操作，当一次循环中没有成功的松弛操作时，算法停止。

每次循环是 $(O(m))$ 的，在最短路存在的情况下，由于一次松弛操作会使最短路的边数至少 $(+1)$ ，而最短路的边数最多为 $(n - 1)$ ，因此整个算法最多执行 $(n - 1)$ 轮松弛操作。故总时间复杂度为 $(O(nm))$ 。

但还有一种情况，如果从 (S) 点出发，抵达一个负环时，松弛操作会无休止地进行下去。注意到前面的论证中已经说明了，对于最短路存在的图，松弛操作最多只会执行 (n - 1) 轮，因此如果第 (n) 轮循环时仍然存在能松弛的边，说明从 (S) 点出发，能够抵达一个负环。

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5  constexpr int INF = 0x3f3f3f3f;
6  constexpr int MAXN = 0;
7
8  int n, m;
9  int u, v, w;
10
11 void violet() {
12     std::cin >> n >> m;
13
14     int tot = 1;
15     std::vector<int> head(n + 1, -1);
16     struct Edge {
17         int to; int nxt; int w;
18     };
19     std::vector<Edge> G(2 * m + 10);
20     auto AddEdge = [&](int u, int v, int w) → void {
21         G[tot].to = v;
22         G[tot].w = w;
23         G[tot].nxt = head[u];
24         head[u] = tot ++;
25     };
26
27     int __ = m;
28     while (__ --) {
29         std::cin >> u >> v >> w;
30         if (w ≥ 0) {
31             AddEdge(u, v, w); AddEdge(v, u, w);
32         } else {
33             AddEdge(u, v, w);
34         }
35     }
36
37     std::vector<int> dis(n + 1, INF);
38     auto Bellman_Ford = [&](int s) → bool {
39         dis[s] = 0;
40         bool ok = false;
41         for (int k = 1; k ≤ n; k ++ ) {
42             ok = false;
43             for (int u = 1; u ≤ n; u ++ ) {
44                 if (dis[u] == INF) continue;
45                 for (int i = head[u]; ~i; i = G[i].nxt) {

```

```

46         int v = G[i].to; int w = G[i].w;
47         if (dis[v] > dis[u] + w) {
48             dis[v] = dis[u] + w;
49             ok = true;
50         }
51     }
52 }
53 if (!ok) {
54     break;
55 }
56 }
57 return ok;
58 };
59
60 if (Bellman_Ford(1)) {
61     std::cout << "YES\n";
62 } else {
63     std::cout << "NO\n";
64 }
65
66 /*
67 Test Sample
68
69 2
70 3 4
71 1 2 2
72 1 3 4
73 2 3 1
74 3 1 -3
75 3 3
76 1 2 3
77 2 3 4
78 3 1 -8
79
80 NO
81 YES
82 */
83 }
84
85 signed main() {
86     std::cin.tie(nullptr) -> sync_with_stdio(false);
87     std::cout.tie(nullptr);
88
89     int _ = 1;
90     for (std::cin >> _; _; _ --)
91         violet();
92
93     return 0;
94 }

```


SPFA

即 *Shortest Path Faster Algorithm* 。

基于 *Bellman-Ford* 的基础，使用队列优化。

很多时候我们并不需要那么多无用的松弛操作。

很显然，只有上一次被松弛的结点，所连接的边，才有可能引起下一次的松弛操作。

那么我们用队列来维护「哪些结点可能会引起松弛操作」，就能只访问必要的边了。

SPFA 也可以用于判断 (s) 点是否能抵达一个负环，只需记录最短路经过了多少条边，当经过了至少 (n) 条边时，说明 (s) 点可以抵达一个负环。

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5  constexpr int INF = 0x3f3f3f3f;
6  constexpr int MAXN = 2e3 + 6;
7
8  int n, m;
9  int u, v, w;
10
11 void violet() {
12     std::cin >> n >> m;
13
14     int tot = 1;
15     std::vector<int> head(n + 1, -1);
16     struct Edge {
17         int to; int nxt; int w;
18     };
19     std::vector<Edge> G(2 * m + 10);
20     auto AddEdge = [&](int u, int v, int w) → void {
21         G[tot].to = v;
22         G[tot].w = w;
23         G[tot].nxt = head[u];
24         head[u] = tot ++;
25     };
26
27     int __ = m;
28     while (__ --) {
29         std::cin >> u >> v >> w;
30         if (w ≥ 0) {
31             AddEdge(u, v, w); AddEdge(v, u, w);
32         } else {
33             AddEdge(u, v, w);
34         }
35     }
36

```

```

37     std::vector<int> dis(n + 1, INF);
38     std::vector<int> cnt(n + 1, 0);
39     bool vis[MAXN] = { false };
40     auto SPFA = [&](int s) → bool {
41         std::queue<int> q;
42
43         dis[s] = 0; vis[s] = 1; q.push(s);
44         while (!q.empty()) {
45             int u = q.front();
46             q.pop(); vis[u] = false;
47             for (int i = head[u]; ~i; i = G[i].nxt) {
48                 int v = G[i].to; int w = G[i].w;
49                 if (dis[v] > dis[u] + w) {
50                     dis[v] = dis[u] + w; cnt[v] = cnt[u] + 1;
51                     if (cnt[v] ≥ n) return false;
52                     if (!vis[v]) {
53                         q.push(v); vis[v] = true;
54                     }
55                 }
56             }
57         }
58         return true;
59     };
60
61     bool ok = SPFA(1);
62     if (ok) {
63         std::cout << "NO\n";
64     } else {
65         std::cout << "YES\n";
66     }
67
68     /*
69     Test Sample
70
71     2
72     3 4
73     1 2 2
74     1 3 4
75     2 3 1
76     3 1 -3
77     3 3
78     1 2 3
79     2 3 4
80     3 1 -8
81
82     NO
83     YES
84     */
85 }
86
87 signed main() {
88     std::cin.tie(nullptr)→sync_with_stdio(false);

```

```

89     std::cout.tie(nullptr);
90
91     int _ = 1;
92     for (std::cin >> _; _; _ --)
93         violet();
94
95     return 0;
96 }

```

虽然在大多数情况下 *SPFA* 跑得很快，但其最坏情况下的时间复杂度为 $O(nm)$ ，将其卡到这个复杂度也是不难的，所以考试时要谨慎使用（在没有负权边时最好使用 *Dijkstra* 算法，在有负权边且题目中的图没有特殊性质时，若 *SPFA* 是 *std* 的一部分，题目不应当给出 *Bellman-Ford* 无法通过的数据范围）。

SPFA 的其他优化

- 堆优化：将队列换成堆，与 *Dijkstra* 的区别是允许一个点多次入队。在有负权边的图可能被卡成指数级复杂度。
- 栈优化：将队列换成栈（即将原来的 *BFS* 过程变成 *DFS*），在寻找负环时可能具有更高效率，但最坏时间复杂度仍然为指数级。
- *LLL* 优化：将普通队列换成双端队列，每次将入队结点距离和队内距离平均值比较，如果更大则插入至队尾，否则插入队首。
- *SLF* 优化：将普通队列换成双端队列，每次将入队结点距离和队首比较，如果更大则插入至队尾，否则插入队首。
- *D'Esopo-Pape* 算法：将普通队列换成双端队列，如果一个节点之前没有入队，则将其插入队尾，否则插入队首。

Dijkstra

Dijkstra 一般是按照下面的步骤来的：

将结点分成两个集合：已确定最短路长度的点集（记为（*S*）集合）和未确定最短路长度的点集（记为（*T*）集合）。一开始所有的点都属于（*T*）集合。

初始化（ $\text{dis}(s) = 0$ ），其他点的（ dis ）均为（ $+\infty$ ）。

然后重复这些操作：

1. 从（*T*）集合中，选取一个最短路长度最小的结点，移到（*S*）集合中。
2. 对那些刚刚被加入（*S*）集合的结点的所有出边执行松弛操作。

直到（*T*）集合为空，算法结束。

朴素的实现方法为每次 (2) 操作执行完毕后，直接在 (T) 集合中暴力寻找最短路长度最小的结点。(2) 操作总时间复杂度为 ($O(m)$)，(1) 操作总时间复杂度为 ($O(n^2)$)，全过程的时间复杂度为 ($O(n^2 + m) = O(n^2)$)。

可以用堆来优化这一过程：每成功松弛一条边 ((u, v))，就将 (v) 插入堆中 (如果 (v) 已经在堆中，直接执行 *Decrease - key*)，(1) 操作直接取堆顶结点即可。共计 ($O(m)$) 次 *Decrease - key*，($O(n)$) 次 *pop*，选择不同堆可以取到不同的复杂度。堆优化能做到的最优复杂度为 ($O(n \log n + m)$)，能做到这一复杂度的有斐波那契堆等。

特别地，可以使用优先队列维护，此时无法执行 *Decrease - key* 操作，但可以通过每次松弛时重新插入该结点，且弹出时检查该结点是否已被松弛过，若是则跳过，复杂度 ($O(m \log n)$)，优点是实现较简单。

这里的堆也可以用线段树来实现，复杂度为 ($O(m \log n)$)，在一些特殊的非递归线段树实现下，该做法常数比堆更小。并且线段树支持的操作更多，在一些特殊图问题上只能用线段树来维护。

在稀疏图中，($m = O(n)$)，堆优化的 *Dijkstra* 算法具有较大的效率优势；而在稠密图中，($m = O(n^2)$)，这时候使用朴素实现更优。

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5  constexpr int INF = 0x3f3f3f3f;
6  const int MAXN = 0;
7
8  int n, m, s;
9  int u, v, w;
10
11 void violet() {
12     std::cin >> n >> m >> s;
13
14     int tot = 1;
15     std::vector<int> head(n + 1, -1);
16     struct Edge {
17         int to; int nxt; int w;
18     };
19     std::vector<Edge> G(2 * m + 10);
20     auto AddEdge = [&](int u, int v, int w) → void {
21         G[tot].to = v;
22         G[tot].w = w;
23         G[tot].nxt = head[u];
24         head[u] = tot ++;
25     };
26
27     int __ = m;
28     while (__ --) {

```

```

29     std::cin >> u >> v >> w;
30     AddEdge(u, v, w);
31 }
32
33 std::vector<int> dis(n + 1, INF);
34 auto Dijkstra = [&](int s) → void {
35     dis[s] = 0;
36     struct node {
37         int id; int d;
38         bool operator < (const node &a) const {
39             return d > a.d;
40         }
41     };
42     std::priority_queue<node> heap;
43     heap.push({s, dis[s]});
44
45     while (!heap.empty()) {
46         auto [u, d] = heap.top();
47         heap.pop();
48
49         if (d > dis[u]) continue;
50         for (int i = head[u]; ~i; i = G[i].nxt) {
51             int v = G[i].to; int w = G[i].w;
52             if (dis[v] > dis[u] + w) {
53                 dis[v] = dis[u] + w;
54                 heap.push({v, dis[v]});
55             }
56         }
57     }
58 };
59
60 Dijkstra(1);
61
62 for (int i = 1; i ≤ n; i++) {
63     std::cout << dis[i] << " \n"[i == n];
64 }
65
66 /*
67 Test Sample
68
69 4 6 1
70 1 2 2
71 2 3 2
72 2 4 1
73 1 3 5
74 3 4 3
75 1 4 4
76
77 0 2 4 3
78 */
79 }
80

```

```

81 signed main() {
82     std::cin.tie(nullptr)→sync_with_stdio(false);
83     std::cout.tie(nullptr);
84
85     int _ = 1;
86     // for (std::cin >> _; _; _ --)
87         violet();
88
89     return 0;
90 }

```

Johnson

Johnson 和 *Floyd* 一样，是一种能求出无负环图上任意两点间最短路径的算法。

任意两点间的最短路可以通过枚举起点，跑 (n) 次 *Bellman-Ford* 算法解决，时间复杂度是 $(O(n^2m))$ 的，也可以直接用 *Floyd* 算法解决，时间复杂度为 $(O(n^3))$ 。

注意到堆优化的 *Dijkstra* 算法求单源最短路径的时间复杂度比 *Bellman-Ford* 更优，如果枚举起点，跑 (n) 次 *Dijkstra* 算法，就可以在 $(O(nm\log m))$ （取决于 *Dijkstra* 算法的实现）的时间复杂度内解决本问题，比上述跑 (n) 次 *Bellman-Ford* 算法的时间复杂度更优秀，在稀疏图上也比 *Floyd* 算法的时间复杂度更加优秀。

但 *Dijkstra* 算法不能正确求解带负权边的最短路，因此我们需要对原图上的边进行预处理，确保所有边的边权均非负。

Johnson 算法则通过另外一种方法来给每条边重新标注边权。

我们新建一个虚拟节点（在这里我们就设它的编号为 (0) ）。从这个点向其他所有点连一条边权为 (0) 的边。

接下来用 *Bellman-Ford* 算法求出从 (0) 号点到其他所有点的最短路，记为 (h_i) 。

假如存在一条从 (u) 点到 (v) 点，边权为 (w) 的边，则我们将该边的边权重新设置为 $(w + h_u - h_v)$ 。

接下来以每个点为起点，跑 (n) 轮 *Dijkstra* 算法即可求出任意两点间的最短路了。

一开始的 *Bellman-Ford* 算法并不是时间上的瓶颈，若使用 *priority_queue* 实现 *Dijkstra* 算法，该算法的时间复杂度是 $(O(nm\log m))$ 。

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5  constexpr int INF = 1e9;
6  const int MAXN = 1e5 + 5;
7

```

```

8  int n, m;
9  int u, v, w;
10
11 void violet() {
12     std::cin >> n >> m;
13
14     int tot = 1;
15     std::vector<int> head(n + 10, -1);
16     struct Edge {
17         int to; int nxt; int w;
18     };
19     std::vector<Edge> G(n + m + 10);
20     auto AddEdge = [&](int u, int v, int w) -> void {
21         G[tot].to = v;
22         G[tot].nxt = head[u];
23         G[tot].w = w;
24         head[u] = tot ++;
25     };
26
27     int __ = m;
28     while (__ --) {
29         std::cin >> u >> v >> w;
30         AddEdge(u, v, w);
31     }
32
33     std::vector<ll> dis1(n + 10, INF);
34     std::vector<ll> cnt(n + 10, 0);
35     bool vis[MAXN] = { false };
36     auto SPFA = [&](int s) -> bool {
37         std::queue<int> q;
38
39         dis1[s] = 0; vis[s] = 1; q.push(s);
40         while (!q.empty()) {
41             int u = q.front();
42             q.pop(); vis[u] = false;
43             for (int i = head[u]; ~i; i = G[i].nxt) {
44                 int v = G[i].to; int w = G[i].w;
45                 if (dis1[v] > dis1[u] + w) {
46                     dis1[v] = dis1[u] + w; cnt[v] = cnt[u] + 1;
47                     if (cnt[v] >= n + 1) return false;
48                     if (!vis[v]) {
49                         q.push(v); vis[v] = true;
50                     }
51                 }
52             }
53         }
54         return true;
55     };
56
57     for (int i = 1; i <= n; i ++) AddEdge(0, i, 0);
58     if (!SPFA(0)) {
59         std::cout << -1 << '\n';

```

```

60     return ;
61 }
62
63 for (int u = 1; u ≤ n; u ++ ) {
64     for (int i = head[u]; ~i; i = G[i].nxt) {
65         int v = G[i].to;
66         G[i].w += dis1[u] - dis1[v];
67         if (G[i].w < 0) G[i].w = 0;
68     }
69 }
70
71 std::vector<ll> dis2(n + 10, INF);
72 auto Dijkstra = [&](int s) → void {
73     dis2.assign(n + 10, INF);
74     dis2[s] = 0;
75     struct node {
76         int id; int d;
77         bool operator < (const node &a) const {
78             return d > a.d;
79         }
80     };
81     std::priority_queue<node> heap;
82     heap.push({s, dis2[s]});
83
84     while (!heap.empty()) {
85         auto [u, d] = heap.top();
86         heap.pop();
87
88         if (d > dis2[u]) continue;
89         for (int i = head[u]; ~i; i = G[i].nxt) {
90             int v = G[i].to; int w = G[i].w;
91             if (dis2[v] > dis2[u] + w) {
92                 dis2[v] = dis2[u] + w;
93                 heap.push({v, dis2[v]});
94             }
95         }
96     }
97 };
98
99 for (int i = 1; i ≤ n; i ++ ) {
100     Dijkstra(i);
101     ll ans = 0;
102     for (int j = 1; j ≤ n; j ++ ) {
103         if (dis2[j] == INF) {
104             ans += 1LL * j * INF;
105         } else {
106             ans += 1LL * j * (dis2[j] + dis1[j] - dis1[i]);
107         }
108     }
109     std::cout << ans << '\n';
110 }
111

```



```
112  /*
113  Test Sample
114  5 7
115  1 2 4
116  1 4 10
117  2 3 7
118  4 5 3
119  4 2 -2
120  3 4 -3
121  5 3 4
122
123  128
124  1000000072
125  999999978
126  1000000026
127  1000000014
128
129
130  5 5
131  1 2 4
132  3 4 9
133  3 4 -3
134  4 5 3
135  5 3 -2
136
137  -1
138  */
139  }
140
141  signed main() {
142      std::cin.tie(nullptr)→sync_with_stdio(false);
143      std::cout.tie(nullptr);
144
145      int _ = 1;
146      // for (std::cin >> _; _; _ --)
147          violet();
148
149      return 0;
150  }
```

比较

最短路算法	Floyd	Bellman-Ford	Dijkstra	Johnson
最短路类型	每对结点之间的最短路	单源最短路	单源最短路	每对结点之间的最短路
作用于	任意图	任意图	非负权图	任意图
能否检测负环?	能	能	不能	能
时间复杂度	($O(N^3)$)	($O(NM)$)	($O(M \log M)$)	($O(NM \log M)$)

输出方案

开一个 *pre* 数组，在更新距离的时候记录下来后面的点是如何转移过去的，算法结束前再递归地输出路径即可。

比如 *Floyd* 就要记录 $pre[i][j] = k;$, *Bellman-Ford* 和 *Dijkstra* 一般记录 $pre[v] = u。$

3.8、生成树

最小生成树

- *Kruskal*

```
1  * @Author: AlgoStruggler
2  * @Create Time: 2026-02-04 16:26:55
3  * "To the Splendor of ACM-ICPC, We Consecrate Our AC Triumphs."
4  */
5  #define _CRT_SECURE_NO_WARNINGS
6  #pragma GCC optimize("O2")
7  #include "bits/stdc++.h"
8  // #include <ext/pb_ds/assoc_container.hpp>
9  // using namespace __gnu_pbds;
10
11 using i64 = long long;
12 using ldb = long double;
13 typedef unsigned long long u64;
14 typedef __int128 i128;
15
16 const double eps = 1e-9;
```

```

17 constexpr int INF = 0x3f3f3f3f;
18 constexpr i64 LINF = 0x3f3f3f3f3f3f3fLL;
19 constexpr i64 mod = 998244353;
20 constexpr i64 MOD = 1e9 + 7;
21 constexpr int MAXN = 0;
22
23 template <typename T>
24 class DisjointSetUnion {
25 private:
26     int N;
27     std::vector<int> Parent;
28     std::vector<int> Siz;
29 public:
30     DisjointSetUnion(int n) : N(n), Parent(N + 1), Siz(N + 1) {
31         std::iota(Parent.begin() + 1, Parent.end(), 1);
32         Siz.assign(N + 1, 1);
33     }
34
35     int find(int x) {
36         while (x != Parent[x]) {
37             x = Parent[x] = Parent[Parent[x]];
38         }
39         return x;
40     }
41
42     bool same(int x, int y) {
43         return find(x) == find(y);
44     }
45
46     bool Merge(int x, int y) {
47         int RootX = find(x);
48         int RootY = find(y);
49
50         if (RootX == RootY) {
51             return false;
52         }
53
54         if (Siz[RootX] < Siz[RootY]) {
55             std::swap(RootX, RootY);
56         }
57
58         Parent[RootY] = RootX;
59         Siz[RootX] += Siz[RootY];
60
61         return true;
62     }
63
64     int size(int x) {
65         return Siz[find(x)];
66     }
67 };
68

```

```

69 static inline void VioletEverGraden() {
70     int n, m;
71     std::cin >> n >> m;
72
73     struct Edge { i64 u; i64 v; i64 w; };
74     std::vector<Edge> G;
75
76     int __ = m;
77     while (__ --) {
78         i64 u, v, w;
79         std::cin >> u >> v >> w;
80         G.push_back({u, v, w});
81     }
82
83     int cnt = 0; i64 ans = 0;
84     auto Kruskal = [&]() -> void {
85         std::sort(G.begin(), G.end(), [&](Edge a, Edge b) -> bool {
86             return a.w < b.w;
87         });
88
89         DisjointSetUnion<int> DSU(n);
90         for (auto [u, v, w] : G) {
91             if (DSU.Merge(u, v)) {
92                 ans += w;
93                 cnt ++;
94                 if (cnt == n - 1) {
95                     break;
96                 }
97             }
98         }
99     };
100
101     Kruskal();
102
103     if (cnt != n - 1) {
104         std::cout << "NO\n";
105     } else {
106         std::cout << ans << '\n';
107     }
108
109     /* 愿幸运女神与最纯粹的前行者同在。 */
110     /*
111     Test Sample
112
113     5 7
114     4 5 2
115     1 3 0
116     1 4 1
117     2 1 1
118     4 1 0
119     2 4 0
120     4 3 0

```

```

121
122 2
123
124 4 2
125 1 2 -12
126 3 4 2
127
128 NO
129
130
131 */
132 }
133
134 signed main() {
135 #ifdef Algostruggler
136     freopen("A.in", "r", stdin);
137 #endif // Algostruggler
138
139     std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);
140     std::cout.tie(nullptr);
141
142     /*std::cout << std::fixed << std::setprecision(9);
143     std::cerr << std::fixed << std::setprecision(9);*/
144
145     int _ = 1;
146     //for (std::cin >> _; _; _--)
147         VioletEverGraden();
148
149     return 0;
150 }
```

四、数据结构

4.1、并查集

```

1  template <typename T>
2  class DisjointSetUnion {
3  private:
4      int N;
5      std::vector<int> Parent;
6      std::vector<int> Size;
7  public:
8      DisjointSetUnion(int n) : N(n), Parent(N + 1), Size(N + 1) {
9          std::iota(Parent.begin() + 1, Parent.end(), 1);
10         Size.assign(N + 1, 1);
11     }
12
13     int find(int x) {
```

```

14     while (x != Parent[x]) {
15         x = Parent[x] = Parent[Parent[x]];
16     }
17     return x;
18 }
19
20 bool same(int x, int y) {
21     return find(x) == find(y);
22 }
23
24 bool Merge(int x, int y) {
25     int RootX = find(x);
26     int RootY = find(y);
27
28     if (RootX == RootY) {
29         return false;
30     }
31
32     if (Size[RootX] < Size[RootY]) {
33         std::swap(RootX, RootY);
34     }
35
36     Parent[RootY] = RootX;
37     Size[RootX] += Size[RootY];
38
39     return true;
40 }
41
42 int size(int x) {
43     return Size[find(x)];
44 }
45 };

```

4.2、树状数组

```

1  template <typename T>
2  class Fenwick {
3  private:
4      int N;
5      std::vector<i64> BIT;
6      inline i64 lowbit(i64 x) {
7          return x & -x;
8      }
9
10     T getSum(int pos) {
11         T res = 0;
12         while (pos > 0) {
13             res += BIT[pos];
14             pos -= lowbit(pos);
15         }

```

```

16         return res;
17     }
18 public:
19     Fenwick(int n = 0) : N(n), BIT(n + 1, 0) {}
20
21     Fenwick(const std::vector<T> &arr) {
22         N = arr.size() - 1;
23         BIT.assign(N + 1, 0);
24
25         std::vector<T> prefix(N + 1, 0);
26         for (int i = 1; i ≤ N; i++) {
27             prefix[i] = prefix[i - 1] + arr[i];
28             BIT[i] = prefix[i] - prefix[i - lowbit(i)];
29         }
30     }
31
32     void Update(int pos, T Val) {
33         while (pos ≤ N) {
34             BIT[pos] += Val;
35             pos += lowbit(pos);
36         }
37     }
38
39     T rangeSum(int L, int R) {
40         return getSum(R) - getSum(L - 1);
41     }
42
43     T getVal(int pos) {
44         return rangeSum(pos, pos);
45     }
46
47     void setVal(int pos, i64 Val) {
48         i64 Cur = getVal(pos);
49         i64 Dif = Val - Cur;
50         Update(pos, Dif);
51     }
52
53     int size() const {
54         return N;
55     }
56
57     void clear() {
58         std::fill(BIT.begin(), BIT.end(), 0);
59     }
60 };

```

4.3、块状数组

```

1  num = sqrt(n);
2  for (int i = 1; i ≤ num; i++)
3      st[i] = n / num * (i - 1) + 1, ed[i] = n / num * i;
4  ed[num] = n;
5  for (int i = 1; i ≤ num; i++) {
6      for (int j = st[i]; j ≤ ed[i]; j++) {
7          belong[j] = i;
8      }
9      size[i] = ed[i] - st[i] + 1;
10 }
11 void Sort(int k) {
12     for (int i = st[k]; i ≤ ed[k]; i++) t[i] = a[i];
13     sort(t + st[k], t + ed[k] + 1);
14 }
15
16 void PushDown(int x) {
17     if (delta[x] ≠ 0×3f3f3f3f3f3f3fll) // 用该值标记块内没有被整体赋值
18         for (int i = st[x]; i ≤ ed[x]; i++) a[i] = t[i] = delta[x];
19     delta[x] = 0×3f3f3f3f3f3f3fll;
20 }
21
22 void Modify(int l, int r, int c) {
23     int x = belong[l], y = belong[r];
24     PushDown(x);
25     if (x == y) {
26         for (int i = l; i ≤ r; i++) a[i] = c;
27         Sort(x);
28         return;
29     }
30     PushDown(y);
31     for (int i = l; i ≤ ed[x]; i++) a[i] = c;
32     for (int i = st[y]; i ≤ r; i++) a[i] = c;
33     Sort(x);
34     Sort(y);
35     for (int i = x + 1; i < y; i++) delta[i] = c;
36 }
37
38 int Binary_Search(int l, int r, int c) {
39     int ans = l - 1, mid;
40     while (l ≤ r) {
41         mid = (l + r) / 2;
42         if (t[mid] ≤ c)
43             ans = mid, l = mid + 1;
44         else
45             r = mid - 1;
46     }
47     return ans;
48 }

```



```

49
50 int Answer(int l, int r, int c) {
51     int ans = 0, x = belong[l], y = belong[r];
52     PushDown(x);
53     if (x == y) {
54         for (int i = l; i ≤ r; i++)
55             if (a[i] ≤ c) ans++;
56         return ans;
57     }
58     PushDown(y);
59     for (int i = l; i ≤ ed[x]; i++)
60         if (a[i] ≤ c) ans++;
61     for (int i = st[y]; i ≤ r; i++)
62         if (a[i] ≤ c) ans++;
63     for (int i = x + 1; i ≤ y - 1; i++) {
64         if (0x3f3f3f3f3f3f3fll == delta[i])
65             ans += Binary_Search(st[i], ed[i], c) - st[i] + 1;
66         else if (delta[i] ≤ c)
67             ans += size[i];
68     }
69     return ans;
70 }

```

4.4、ST表

```

1  template <typename Info, typename T>
2  class SparseTable {
3  private:
4      int N;
5      std::vector<std::vector<Info>> ST;
6      std::vector<int> log2;
7  public:
8      SparseTable() : N(0) {}
9
10     SparseTable(const std::vector<T> &arr) {
11         N = (int)arr.size() - 1;
12
13         log2.resize(N + 1);
14         log2[1] = 0;
15         for (int i = 2; i ≤ N; i++) {
16             log2[i] = log2[i / 2] + 1;
17         }
18
19         int K = log2[N] + 1;
20         ST.resize(K, std::vector<Info>(N + 1));
21
22         for (int i = 1; i ≤ N; i++) {
23             ST[0][i] = Info(arr[i]);

```

```

24     }
25     for (int j = 1; j < K; j++) {
26         for (int i = 1; i + (1 << j) - 1 ≤ N; i++) {
27             ST[j][i] = ST[j - 1][i] + ST[j - 1][i + (1 << (j - 1))];
28         }
29     }
30 }
31
32 Info Query(int l, int r) {
33     int j = log2[r - l + 1];
34     Info res = ST[j][l] + ST[j][r - (1 << j) + 1];
35     return res;
36 }
37 };
38 struct Info {
39     i64 max;
40     i64 min;
41     i64 gcd;
42     Info() :
43         max(std::numeric_limits<i64>::min()),
44         min(std::numeric_limits<i64>::max()),
45         gcd(0) {}
46     Info(i64 val) :
47         max(val),
48         min(val),
49         gcd(val) {}
50 };
51
52 Info operator + (const Info &a, const Info &b) {
53     Info res;
54     res.max = std::max(a.max, b.max);
55     res.min = std::min(a.min, b.min);
56     res.gcd = std::gcd(a.gcd, b.gcd);
57     return res;
58 }

```

4.5、线段树

快速线段树（单点修改+区间最值）

```

1  struct Segt {
2      vector<int> w;
3      int n;
4      Segt(int n) : w(2*n, (int)-2E9), n(n) {}
5      void modify(int pos, int val) {
6          for (w[pos += n] = val; pos > 1; pos /= 2)
7              w[pos/2] = max(w[pos], w[pos^1]);
8      }
9      int ask(int l, int r) {
10         int res = -2E9;

```

```

11         for (l += n, r += n; l < r; l /= 2, r /= 2) {
12             if (l & 1) res = max(res, w[l++]);
13             if (r & 1) res = max(res, w[--r]);
14         }
15         return res;
16     }
17 };

```

函数线段树

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  #define rep(i, l, r, x) for (int i = l; i ≤ r; i += x)
5
6  const int MAXN = 1e5 + 5;
7
8  int n, m;
9  std::vector<ll> a;
10 int op; ll x, y, k;
11
12 struct info {
13     ll sum;
14 };
15 std::vector<info> seg((MAXN << 2) + 10);
16 std::vector<ll> tag((MAXN << 2) + 10);
17
18 void update(ll id) {
19     seg[id].sum = seg[id * 2].sum + seg[id * 2 + 1].sum;
20 }
21
22 void build(ll id, ll l, ll r) {
23     tag[id] = 0;
24     if (l == r) {
25         seg[id].sum = a[l];
26     } else {
27         ll mid = l + r >> 1;
28         build(id * 2, l, mid);
29         build(id * 2 + 1, mid + 1, r);
30         update(id);
31     }
32 }
33
34 void addtag(ll id, ll l, ll r, ll val) {
35     tag[id] += val;
36     seg[id].sum += val * (r - l + 1);
37 }
38
39 void pushdown(ll id, ll l, ll r) {
40     if (tag[id]) {

```

```

41     ll mid = l + r >> 1;
42     addtag(id * 2, l, mid, tag[id]);
43     addtag(id * 2 + 1, mid + 1, r, tag[id]);
44     tag[id] = 0;
45 }
46 }
47
48 void modify(ll id, ll l, ll r, ll ql, ll qr, ll val) {
49     if (l ≥ ql && r ≤ qr) {
50         addtag(id, l, r, val);
51         return;
52     }
53     pushdown(id, l, r);
54     ll mid = l + r >> 1;
55     if (ql ≤ mid) {
56         modify(id * 2, l, mid, ql, qr, val);
57     }
58     if (qr > mid) {
59         modify(id * 2 + 1, mid + 1, r, ql, qr, val);
60     }
61     update(id);
62 }
63
64 ll query(ll id, ll l, ll r, ll ql, ll qr) {
65     if (ql ≤ l && r ≤ qr) {
66         return seg[id].sum;
67     }
68     pushdown(id, l, r);
69     ll mid = l + r >> 1;
70     ll res = 0;
71     if (ql ≤ mid) {
72         res += query(id * 2, l, mid, ql, qr);
73     }
74     if (qr > mid) {
75         res += query(id * 2 + 1, mid + 1, r, ql, qr);
76     }
77     return res;
78 }
79
80 void violet() {
81     std::cin >> n >> m;
82     a.resize(n + 10);
83     for (int i = 1; i ≤ n; i++) {
84         std::cin >> a[i];
85     }
86
87     build(1, 1, n);
88
89     int __ = m;
90     while (__--) {
91         std::cin >> op;
92         if (op == 1) {

```

```

93         std::cin >> x >> y >> k;
94         modify(1, 1, n, x, y, k);
95     } else {
96         std::cin >> x >> y;
97         ll ans = query(1, 1, n, x, y);
98         std::cout << ans << '\n';
99     }
100 }
101 }
102
103 signed main() {
104     std::cin.tie(nullptr) -> sync_with_stdio(false);
105     std::cout.tie(nullptr);
106
107     int _ = 1;
108     // for (std::cin >> _; _; _ --)
109         violet();
110
111     return 0;
112 }
113
114 /*
115 Test Sample
116 5 5
117 1 5 4 2 3
118 2 2 4
119 1 2 3 2
120 2 3 4
121 1 1 5 1
122 2 1 4
123
124 11
125 8
126 20
127 */

```

线段树封装类

```

1  template <typename Info, typename Tag, typename T = i64>
2  class LazySegmentTree {
3  private:
4      int N;
5      std::vector<i64> a;
6      std::vector<Info> Seg;
7      std::vector<Tag> Lazy;
8
9      void push_up(i64 id) {
10         Seg[id] = Seg[id << 1] + Seg[id << 1 | 1];
11     }
12

```

```

13 void Build(i64 id, i64 l, i64 r) {
14     Lazy[id] = Tag();
15     if (l == r) {
16         Seg[id] = Info(a[l], 1);
17     } else {
18         i64 mid = l + r >> 1;
19         Build(id << 1, l, mid);
20         Build(id << 1 | 1, mid + 1, r);
21         push_up(id);
22     }
23 }
24
25 void ApplyTag(i64 id, const Tag &t) {
26     Seg[id].ApplyTag(t);
27     Lazy[id].ApplyTag(t);
28 }
29
30 void push_down(i64 id) {
31     ApplyTag(id << 1, Lazy[id]);
32     ApplyTag(id << 1 | 1, Lazy[id]);
33     Lazy[id] = Tag();
34 }
35
36 void Update(i64 id, i64 l, i64 r, i64 pos, const Tag &v) {
37     if (l == r) {
38         ApplyTag(id, v);
39         return;
40     }
41     push_down(id);
42
43     i64 mid = l + r >> 1;
44     if (pos <= mid) {
45         Update(id << 1, l, mid, pos, v);
46     } else {
47         Update(id << 1 | 1, mid + 1, r, pos, v);
48     }
49     push_up(id);
50 }
51
52 void Modify(i64 id, i64 l, i64 r, i64 ql, i64 qr, const Tag &v) {
53     if (l >= ql and r <= qr) {
54         ApplyTag(id, v);
55         return ;
56     }
57     push_down(id);
58
59     i64 mid = l + r >> 1;
60     if (ql <= mid) {
61         Modify(id << 1, l, mid, ql, qr, v);
62     }
63     if (qr > mid) {
64         Modify(id << 1 | 1, mid + 1, r, ql, qr, v);

```

```

65     }
66     push_up(id);
67 }
68
69 Info Query(i64 id, i64 l, i64 r, i64 ql, i64 qr) {
70     if (ql ≤ l and qr ≥ r) {
71         return Seg[id];
72     }
73     push_down(id);
74
75     i64 mid = l + r >> 1;
76     if (qr ≤ mid) {
77         return Query(id << 1, l, mid, ql, qr);
78     }
79     if (ql > mid) {
80         return Query(id << 1 | 1, mid + 1, r, ql, qr);
81     }
82     return Query(id << 1, l, mid, ql, qr) + Query(id << 1 | 1, mid + 1,
r, ql, qr);
83 }
84 public:
85     LazySegmentTree(const std::vector<T> &arr) {
86         a = arr;
87         N = (int)arr.size() - 1;
88         Seg.resize(N << 2);
89         Lazy.resize(N << 2);
90         Build(1, 1, N);
91     }
92
93     void Update(i64 pos, const Tag &v) {
94         return Update(1, 1, N, pos, v);
95     }
96
97     void Modify(i64 l, i64 r, const Tag &v) {
98         return Modify(1, 1, N, l, r, v);
99     }
100
101     Info Query(i64 l, i64 r) {
102         return Query(1, 1, N, l, r);
103     }
104 };
105
106 struct Tag {
107     i64 add;
108     Tag(i64 a = 0) : add(a) {}
109
110     void ApplyTag(const Tag &t) {
111         add += t.add;
112     }
113 };
114
115 struct Info {

```

```

116     i64 sum;
117     i64 len;
118     Info(i64 s = 0, i64 l = 1) : sum(s), len(l) {}
119
120     Info operator + (const Info &a) const {
121         Info res;
122         res.sum = sum + a.sum;
123         // res.max = std::max(max, a.max)
124         res.len = len + a.len;
125         return res;
126     }
127
128     void ApplyTag(const Tag &t) {
129         sum += t.add * len;
130     }
131 };

```

动态开点的线段树

```

1  #include <bits/stdc++.h>
2
3  using ll = long long;
4  const int MAXN = 1e5 + 5;
5
6  int n, m;
7  std::vector<ll> a;
8  int op; ll x, y, k;
9
10 struct Node {
11     ll sum, tag;
12     int ls, rs;
13     Node() : sum(0), tag(0), ls(0), rs(0) {}
14 };
15
16 int root, cnt;
17 std::vector<Node> seg((MAXN << 2) + 10);
18
19 int newNode() {
20     cnt++;
21     if (cnt ≥ seg.size()) {
22         seg.resize(seg.size() * 2);
23     }
24     seg[cnt] = Node();
25     return cnt;
26 }
27
28 void update(int id) {
29     seg[id].sum = seg[seg[id].ls].sum + seg[seg[id].rs].sum;
30 }
31

```



```

32 void build(int &id, ll l, ll r) {
33     if (!id) id = newNode();
34     if (l == r) {
35         seg[id].sum = a[l];
36     } else {
37         ll mid = l + r >> 1;
38         build(seg[id].ls, l, mid);
39         build(seg[id].rs, mid + 1, r);
40         update(id);
41     }
42 }
43
44 void addtag(int id, ll l, ll r, ll val) {
45     if (!id) return;
46     seg[id].tag += val;
47     seg[id].sum += val * (r - l + 1);
48 }
49
50 void pushdown(int id, ll l, ll r) {
51     if (seg[id].tag and l < r) {
52         ll mid = l + r >> 1;
53
54         if (!seg[id].ls) {
55             seg[id].ls = newNode();
56         }
57         if (!seg[id].rs) {
58             seg[id].rs = newNode();
59         }
60
61         addtag(seg[id].ls, l, mid, seg[id].tag);
62         addtag(seg[id].rs, mid + 1, r, seg[id].tag);
63         seg[id].tag = 0;
64     }
65 }
66
67 void modify(int id, ll l, ll r, ll ql, ll qr, ll val) {
68     if (ql ≤ l and r ≤ qr) {
69         addtag(id, l, r, val);
70         return ;
71     }
72     pushdown(id, l, r);
73     ll mid = l + r >> 1;
74
75     if (ql ≤ mid) {
76         if (!seg[id].ls) seg[id].ls = newNode();
77         modify(seg[id].ls, l, mid, ql, qr, val);
78     }
79     if (qr > mid) {
80         if (!seg[id].rs) seg[id].rs = newNode();
81         modify(seg[id].rs, mid + 1, r, ql, qr, val);
82     }
83     update(id);

```

```

84 }
85
86 ll query(int id, ll l, ll r, ll ql, ll qr) {
87     if (!id) {
88         return 0;
89     }
90     if (ql ≤ l and r ≤ qr) {
91         return seg[id].sum;
92     }
93     pushdown(id, l, r);
94     ll mid = l + r >> 1;
95     ll res = 0;
96     if (ql ≤ mid) {
97         res += query(seg[id].ls, l, mid, ql, qr);
98     }
99     if (qr > mid) {
100         res += query(seg[id].rs, mid + 1, r, ql, qr);
101     }
102     return res;
103 }
104
105 void violet() {
106     std::cin >> n >> m;
107     a.resize(n + 10);
108     for (int i = 1; i ≤ n; i++) {
109         std::cin >> a[i];
110     }
111
112     root = 0; cnt = 0;
113     build(root, 1, n);
114
115     int __ = m;
116     while (__--) {
117         std::cin >> op;
118         if (op == 1) {
119             std::cin >> x >> y >> k;
120             modify(root, 1, n, x, y, k);
121         } else {
122             std::cin >> x >> y;
123             ll ans = query(root, 1, n, x, y);
124             std::cout << ans << '\n';
125         }
126     }
127 }
128
129 signed main() {
130     std::cin.tie(nullptr)→sync_with_stdio(false);
131     std::cout.tie(nullptr);
132
133     int _ = 1;
134     // for (std::cin >> _; _; _ --)
135     violet();

```

```

136     return 0;
137 }
138
139 /*
140 Test Sample
141 5 5
142 1 5 4 2 3
143 2 2 4
144 1 2 3 2
145 2 3 4
146 1 1 5 1
147 2 1 4
148
149 11
150 8
151 20
152 */

```

4.6、平衡树

4.7、珂朵莉树

区间赋值的数据结构都可以写分，在数据随机的情况下，复杂度可以保证，时间复杂度： $O(N \log \log N)$ 。

```

1 struct DOT {
2     struct node {
3         int l, r;
4         mutable LL v;
5         node(int l, int r = -1, LL v = 0) : l(l), r(r), v(v) {}
6         bool operator<(const node &o) const {
7             return l < o.l;
8         }
9     };
10    set<Node> s;
11    DOT() {
12        s.clear();
13    }
14
15    auto split(int pos) {
16        auto it = s.lower_bound(node(pos));
17        if (it != s.end() && it->l == pos) return it;
18        it--;
19        int l = it->l, r = it->r;
20        LL v = it->v;

```

```

21     s.erase(it);
22     s.insert(node(l, pos - 1, v));
23     return s.insert(node(pos, r, v)).first;
24 }
25
26 void assign(int l, int r, LL x) {
27     auto itr = split(r + 1), itl = split(l);
28     s.erase(itl, itr);
29     s.insert(node(l, r, x));
30 }
31
32 void add(int l, int r, LL x) {
33     auto itr = split(r + 1), itl = split(l);
34     for (auto it = itl; it != itr; it++) {
35         it->v += x;
36     }
37 }
38
39 LL kth(int l, int r, int k) {
40     vector<pair<LL, int>> a;
41     auto itr = split(r + 1), itl = split(l);
42     for (auto it = itl; it != itr; it++) {
43         a.push_back(pair<LL, int>(it->v, it->r - it->l + 1));
44     }
45     sort(a.begin(), a.end());
46     for (auto [val, len] : a) {
47         k -= len;
48         if (k <= 0) return val;
49     }
50 }
51
52 LL power(LL a, int b, int mod) {
53     a %= mod;
54     LL res = 1;
55     for (; b; b /= 2, a = a * a % mod) {
56         if (b % 2) {
57             res = res * a % mod;
58         }
59     }
60     return res;
61 }
62
63 LL powsum(int l, int r, int x, int mod) {
64     auto itr = split(r + 1), itl = split(l);
65     LL ans = 0;
66     for (auto it = itl; it != itr; it++) {
67         ans = (ans + power(it->v, x, mod) * (it->r - it->l + 1) % mod) %
mod;
68     }
69     return ans;
70 }
71 };

```

4.8、线性基

```

1  template <typename T>
2  class LinearBasis {
3  private:
4      static constexpr int BASE = sizeof(T) * 8;
5      std::vector<T> Basis;
6      std::vector<T> Reduced_Basis;
7      std::vector<u64> Basis_Mask;
8      std::vector<int> Index;
9
10     int Reduce_Size;
11     bool Has_Zoro;
12 public:
13     LinearBasis() {
14         Basis.assign(BASE, T{0});
15         Reduced_Basis.assign(BASE, T{0});
16         Basis_Mask.assign(BASE, 0);
17         Reduce_Size = 0;
18         Has_Zoro = false;
19     }
20
21     bool Insert(T Val, int Id) {
22         u64 Cur = 0;
23         for (int i = BASE - 1; i ≥ 0; i --) {
24             if ((Val >> i) & 1) {
25                 if (!Basis[i]) {
26                     Basis[i] = Val;
27                     Basis_Mask[i] = Cur | (1ULL << Index.size());
28                     Index.push_back(Id);
29                     return true;
30                 }
31
32                 Val ^= Basis[i];
33                 Cur ^= Basis_Mask[i];
34             }
35         }
36         Has_Zoro = true;
37         return false;
38     }
39
40     void ReBuild() {
41         for (int i = BASE - 1; i ≥ 0; i --) {
42             if (Basis[i]) {
43                 for (int j = i - 1; j ≥ 0; j --) {
44                     if ((Basis[i] >> j) & 1) {
45                         Basis[i] ^= Basis[j];
46                         Basis_Mask[i] ^= Basis_Mask[j];
47                     }

```

```

48         }
49     }
50 }
51
52 Reduce_Size = 0;
53 for (int i = 0; i < BASE; i++) {
54     if (Basis[i]) {
55         Reduced_Basis[Reduce_Size++] = Basis[i];
56     }
57 }
58 }
59
60 T Kth_Value(int64 K) {
61     if (Has_Zero) {
62         K--;
63     }
64     if (K == 0) {
65         return T{0};
66     }
67     if (K >= (1ULL << Reduce_Size)) {
68         return T{-1};
69     }
70
71     T res = 0;
72     for (int i = 0; i < Reduce_Size; i++) {
73         if ((K >> i) & 1) {
74             res ^= Reduced_Basis[i];
75         }
76     }
77
78     return res;
79 }
80
81 T Max_Value() const {
82     T res = 0;
83     for (int i = BASE - 1; i >= 0; i--) {
84         if ((res ^ Basis[i]) > res) {
85             res ^= Basis[i];
86         }
87     }
88
89     return res;
90 }
91
92 T Min_Value() const {
93     if (Has_Zero) {
94         return T{0};
95     }
96
97     for (int i = 0; i < BASE; i++) {
98         if (Basis[i]) {
99             return Basis[i];

```

```

100     }
101 }
102
103     return T{0};
104 }
105
106 std::pair<bool, u64> Represent(T Val) const {
107     u64 res = 0;
108     for (int i = BASE - 1; i ≥ 0; i --) {
109         if ((Val >> i) & 1) {
110             if (!Basis[i]) {
111                 return {false, 0ULL};
112             }
113             Val ^= Basis[i];
114             res ^= Basis_Mask[i];
115         }
116     }
117
118     return {true, res};
119 }
120
121 std::vector<int> getId(u64 S) const {
122     std::vector<int> res;
123     for (int i = 0; i < (int)Index.size(); i ++) {
124         if (S & (1ULL << i)) {
125             res.push_back(Index[i]);
126         }
127     }
128
129     return res;
130 }
131
132 int Rank() const {
133     int RK = 0;
134     for (int i = 0; i < BASE; i ++) {
135         if (Basis[i]) {
136             RK ++;
137         }
138     }
139
140     return RK;
141 }
142
143 bool Zero() const {
144     return Has_Zoro;
145 }
146 };

```

4.9、自动取模类

```

1  template <typename T>
2  constexpr T ksm(T a, u64 b) {
3      T res = 1;
4      while (b) {
5          if (b & 1) {
6              res *= a;
7          }
8          a *= a;
9          b >>= 1;
10     }
11     return res;
12 }
13
14 constexpr i64 ksc(i64 a, i64 b, i64 p) {
15 #ifdef __SIZEOF_INT128__
16     return (i64)((__int128)a * b % p);
17 #else
18     i64 res = a * b - i64(1.L * a * b / p - 0.5L) * p;
19     res %= p;
20     if (res < 0) {
21         res += p;
22     }
23     return res;
24 #endif
25 }
26
27 template <i64 P>
28 class MLong {
29 private:
30     i64 x;
31     static i64 Mod;
32
33     constexpr static i64 getMod() {
34         if constexpr (P > 0) {
35             return P;
36         } else {
37             return Mod;
38         }
39     }
40
41     constexpr i64 Normalize(i64 Val) const {
42         i64 m = getMod();
43         Val %= m;
44         if (Val < 0) {
45             Val += m;
46         }
47         return Val;
48     }

```



```

49 public:
50     constexpr MLong() : x(0) {}
51     constexpr MLong(i64 x) : x{Normalize(x % getMod())} {}
52
53     constexpr static void setMod(i64 _Mod) {
54         Mod = _Mod;
55     }
56
57     constexpr i64 val() const {
58         return x;
59     }
60
61     explicit constexpr operator i64() const {
62         return x;
63     }
64
65     constexpr MLong inv() const {
66         assert(x  $\neq$  0);
67         return ksm(*this, getMod() - 2);
68     }
69
70     constexpr MLong pow(u64 b) const {
71         return ksm(*this, b);
72     }
73
74     constexpr MLong operator - () const {
75         return MLong(-x);
76     }
77
78     constexpr MLong &operator *= (MLong rhs) & {
79         x = ksc(x, rhs.x, getMod());
80         return *this;
81     }
82     constexpr MLong &operator += (MLong rhs) & {
83         x = Normalize(x + rhs.x);
84         return *this;
85     }
86     constexpr MLong &operator -= (MLong rhs) & {
87         x = Normalize(x - rhs.x);
88         return *this;
89     }
90     constexpr MLong &operator /= (MLong rhs) & {
91         return *this *= rhs.inv();
92     }
93
94     constexpr MLong& operator ++ () {
95         x = Normalize(x + 1);
96         return *this;
97     }
98     constexpr MLong operator ++ (int) {
99         MLong tmp = *this;
100         ++ (*this);

```

```

101     return tmp;
102 }
103
104 constexpr MLong& operator -- () {
105     x = Normalize(x - 1);
106     return *this;
107 }
108 constexpr MLong operator -- (int) {
109     MLong tmp = *this;
110     -- (*this);
111     return tmp;
112 }
113
114 friend constexpr MLong operator * (MLong lhs, MLong rhs) {
115     MLong res = lhs;
116     res *= rhs;
117     return res;
118 }
119 friend constexpr MLong operator + (MLong lhs, MLong rhs) {
120     MLong res = lhs;
121     res += rhs;
122     return res;
123 }
124 friend constexpr MLong operator - (MLong lhs, MLong rhs) {
125     MLong res = lhs;
126     res -= rhs;
127     return res;
128 }
129 friend constexpr MLong operator / (MLong lhs, MLong rhs) {
130     MLong res = lhs;
131     res /= rhs;
132     return res;
133 }
134
135 friend constexpr bool operator == (MLong lhs, MLong rhs) {
136     return lhs.val() == rhs.val();
137 }
138 friend constexpr bool operator != (MLong lhs, MLong rhs) {
139     return lhs.val() != rhs.val();
140 }
141
142 friend constexpr bool operator < (MLong lhs, MLong rhs) {
143     return lhs.val() < rhs.val();
144 }
145 friend constexpr bool operator ≤ (MLong lhs, MLong rhs) {
146     return lhs.val() ≤ rhs.val();
147 }
148
149 friend constexpr bool operator > (MLong lhs, MLong rhs) {
150     return lhs.val() > rhs.val();
151 }
152 friend constexpr bool operator ≥ (MLong lhs, MLong rhs) {

```

```

153         return lhs.val() ≥ rhs.val();
154     }
155
156     friend constexpr std::istream &operator >> (std::istream &is, MLong &a)
157     {
158         i64 v;
159         is >> v;
160         a = MLong(v);
161         return is;
162     }
163     friend constexpr std::ostream &operator << (std::ostream &os, const
164     MLong &a) {
165         return os << a.val();
166     }
167 };
168
169 template<int P>
170 class MInt {
171 private:
172     int x;
173     static int Mod;
174
175     constexpr static int getMod() {
176         if constexpr (P > 0) {
177             return P;
178         } else {
179             return Mod;
180         }
181     }
182
183     constexpr int Normalize(i64 Val) const {
184         int m = getMod();
185         Val %= m;
186         if (Val < 0) {
187             Val += m;
188         }
189         return Val;
190     }
191 public:
192     constexpr MInt() : x(0) {}
193     constexpr MInt(i64 x) : x{Normalize(x % getMod())} {}
194
195     constexpr static void setMod(i64 _Mod) {
196         Mod = _Mod;
197     }
198
199     constexpr int val() const {
200         return x;
201     }
202
203     explicit constexpr operator int() const {
204         return x;

```

```

203     }
204
205     constexpr MInt inv() const {
206         assert(x  $\neq$  0);
207         return ksm(*this, getMod() - 2);
208     }
209
210     constexpr MInt pow(u64 b) const {
211         return ksm(*this, b);
212     }
213
214     constexpr MInt operator - () const {
215         return MInt(-x);
216     }
217
218     constexpr MInt &operator *= (MInt rhs) & {
219         x = ksc(x, rhs.x, getMod());
220         return *this;
221     }
222     constexpr MInt &operator += (MInt rhs) & {
223         x = Normalize(x + rhs.x);
224         return *this;
225     }
226     constexpr MInt &operator -= (MInt rhs) & {
227         x = Normalize(x - rhs.x);
228         return *this;
229     }
230     constexpr MInt &operator /= (MInt rhs) & {
231         return *this *= rhs.inv();
232     }
233
234     constexpr MInt& operator ++ () {
235         x = Normalize(x + 1);
236         return *this;
237     }
238     constexpr MInt operator ++ (int) {
239         MInt tmp = *this;
240         ++ (*this);
241         return tmp;
242     }
243
244     constexpr MInt& operator -- () {
245         x = Normalize(x - 1);
246         return *this;
247     }
248     constexpr MInt operator -- (int) {
249         MInt tmp = *this;
250         -- (*this);
251         return tmp;
252     }
253
254     friend constexpr MInt operator * (MInt lhs, MInt rhs) {

```

```

255     MInt res = lhs;
256     res *= rhs;
257     return res;
258 }
259 friend constexpr MInt operator + (MInt lhs, MInt rhs) {
260     MInt res = lhs;
261     res += rhs;
262     return res;
263 }
264 friend constexpr MInt operator - (MInt lhs, MInt rhs) {
265     MInt res = lhs;
266     res -= rhs;
267     return res;
268 }
269 friend constexpr MInt operator / (MInt lhs, MInt rhs) {
270     MInt res = lhs;
271     res /= rhs;
272     return res;
273 }
274
275 friend constexpr bool operator == (MInt lhs, MInt rhs) {
276     return lhs.val() == rhs.val();
277 }
278 friend constexpr bool operator != (MInt lhs, MInt rhs) {
279     return lhs.val() != rhs.val();
280 }
281
282 friend constexpr bool operator < (MInt lhs, MInt rhs) {
283     return lhs.val() < rhs.val();
284 }
285 friend constexpr bool operator ≤ (MInt lhs, MInt rhs) {
286     return lhs.val() ≤ rhs.val();
287 }
288
289 friend constexpr bool operator > (MInt lhs, MInt rhs) {
290     return lhs.val() > rhs.val();
291 }
292 friend constexpr bool operator ≥ (MInt lhs, MInt rhs) {
293     return lhs.val() ≥ rhs.val();
294 }
295
296 friend constexpr std::istream &operator >> (std::istream &is, MInt &a)
297 {
298     i64 v;
299     is >> v;
300     a = MInt(v);
301     return is;
302 }
303 friend constexpr std::ostream &operator << (std::ostream &os, const
MInt &a) {
304     return os << a.val();
305 }

```

```

305 };
306
307 template <
308 i64 MLong<0>::Mod = i64(1E18) + 9;
309 template <
310 int MInt<0>::Mod = 998244353;
311
312 using Z998244353 = MInt<998244353>;
313 using Z1e9_7 = MInt<1000000007>;
314 using Z1e9_9 = MInt<1000000009>;
315
316 constexpr auto inv2_998244353 = Z998244353(2).inv();
317 constexpr auto inv2_1e9_7 = Z1e9_7(2).inv();
318 constexpr auto inv2_1e9_9 = Z1e9_9(2).inv();
319
320 using DynZ = MInt<0>;
321 using Z = Z1e9_7;

```

4.10、大整数类

```

1  constexpr int N = 1000;
2
3  struct BigInt {
4      int a[N];
5      BigInt(int x = 0) : a{} {
6          for (int i = 0; x; i++) {
7              a[i] = x % 10;
8              x /= 10;
9          }
10     }
11     BigInt &operator*=(int x) {
12         for (int i = 0; i < N; i++) {
13             a[i] *= x;
14         }
15         for (int i = 0; i < N - 1; i++) {
16             a[i + 1] += a[i] / 10;
17             a[i] %= 10;
18         }
19         return *this;
20     }
21     BigInt &operator/=(int x) {
22         for (int i = N - 1; i ≥ 0; i--) {
23             if (i) {
24                 a[i - 1] += a[i] % x * 10;
25             }
26             a[i] /= x;
27         }
28         return *this;
29     }
30     BigInt &operator+=(const BigInt &x) {

```

```

31         for (int i = 0; i < N; i++) {
32             a[i] += x.a[i];
33             if (a[i] ≥ 10) {
34                 a[i + 1] += 1;
35                 a[i] -= 10;
36             }
37         }
38         return *this;
39     }
40 };
41
42 std::ostream &operator<<(std::ostream &o, const BigInt &a) {
43     int t = N - 1;
44     while (a.a[t] == 0) {
45         t--;
46     }
47     for (int i = t; i ≥ 0; i--) {
48         o << a.a[i];
49     }
50     return o;
51 }

```

五、数论

5.1、常见数列

调和级数

满足调和级数 $O\left(\frac{N}{1} + \frac{N}{2} + \frac{N}{3} + \cdots + \frac{N}{N}\right)$ ，可以用 $N \ln N$ 来拟合，但是会略小，误差量级在 10% 左右。本地可以在 500ms 内完成 10^8 量级的预处理计算。

N的量级	累加和
1	27
2	482
3	7'069
4	93'668
5	1'166'750
6	13'970'034
7	162'725'364
8	1'857'511'568
9	20'877'697'634

素数密度与分布

N的量级	1	2	3	4	5	6	7	8	9
素数数量	4	25	168	1'229	9'592	78'498	664'579	5'761'455	50'847'534

对于任意两个相邻的素数 $p_1, p_2 \leq 10^9$, 有 $|p_1 - p_2| < 300$ 。

因数最多数字与其因数数量

N的量级	1	2	3	4	5	6	7
因数最多数字的因数数量	1	2	4	6	12	24	48
因数最多的数字	-	-	-	7560, 9240	83160, 98280	720720, 831600, 942480, 982800, 997920	-

5.2、素数筛

埃氏筛

欧拉筛

```

1  auto eulerSieve = [&](int n) → void {
2      prime.clear();
3      isPrime.assign(n + 1, true);    // 初始都认为是素数
4      isPrime[0] = isPrime[1] = false;
5
6      for (int i = 2; i ≤ n; ++i) {
7          if (isPrime[i]) {
8              prime.push_back(i);
9          }
10         for (int j = 0; j < (int)prime.size() && i * prime[j] ≤ n; ++j) {
11             isPrime[i * prime[j]] = false;    // 标记为合数
12             if (i % prime[j] == 0) break;    // 核心优化：保证每个合数只被最小
           质因子筛一次
13         }
14     }
15 };

```

5.3、龟速乘

借助浮点数实现

```

1  int mul(int a, int b, int m) {
2      int r = a * b - m * (int)(1.0L / m * a * b);
3      return r - m * (r ≥ m) + m * (r < 0);
4  }

```

借助int128实现

```

1  int mul(int a, int b, int m) {
2      return (int128)a * b % m;
3  }

```

5.4、裴蜀定理

$ax + by = c$ 有整数解的充要条件是 $\gcd(a, b) \mid c$ 。

```

1 LL n, a, ans;
2 LL gcd(LL a, LL b) { return b ? gcd(b, a % b) : a; }
3 int main() {
4     cin >> n;
5     for (int i = 0; i < n; i++) {
6         cin >> a;
7         if (a < 0) a = -a;
8         ans = gcd(ans, a);
9     }
10    cout << ans << "\n";
11 }

```

5.5、逆元

费马小定理

单次计算的复杂度即为快速幂的复杂度 $O(\log X)$ 。限制： MOD 必须是质数，且需要满足 x 与 MOD 互质。

```

1 i64 ksm(i64 a, i64 b, i64 p) {
2     i64 res = 1;
3     while (b) {
4         if (b & 1) {
5             res = res * a % p;
6         }
7         a = a * a % p;
8         b >>= 1;
9     }
10    return res;
11 }
12
13 i64 inv(i64 x, i64 p) {
14     return ksm(x, p - 2, p);
15 }

```

拓展欧几里得

此方法的 MOD 没有限制，复杂度为 $O(\log X)$ ，但是比快速幂法常数大一些。

```

1  int exgcd(int a, int b, int &x, int &y) {
2      if (b == 0) { x = 1, y = 0; return a; }
3      int r = exgcd(b, a % b, x, y);
4      int temp = y;
5      y = x - (a / b) * y;
6      x = temp;
7      return r;
8  }
9  LL getInv(int a, int mod) {
10     LL x, y, d = exgcd(a, mod, x, y);
11     return d == 1 ? (x % mod + mod) % mod : -1;
12 }

```

线性递推

以 $O(N)$ 的复杂度完成 $1 - N$ 中全部逆元的计算。

```

1  inv[1] = 1;
2  for (int i = 2; i ≤ n; i++)
3      inv[i] = (p - p / i) * inv[p % i] % p;

```

阶乘逆元（用于组合数）

```

1  vector<long long> fact, invfact;
2  void precomputeFactorials(int n, long long mod) {
3      fact.resize(n + 1);
4      invfact.resize(n + 1);
5      fact[0] = 1;
6      for (int i = 1; i ≤ n; ++i)
7          fact[i] = fact[i - 1] * i % mod;
8      invfact[n] = qpow(fact[n], mod - 2, mod); // 费马
9      for (int i = n; i ≥ 1; --i)
10         invfact[i - 1] = invfact[i] * i % mod;
11 }
12
13 long long C(int n, int k, long long mod) {
14     if (k < 0 || k > n) return 0;
15     return fact[n] * invfact[k] % mod * invfact[n - k] % mod;
16 }

```

5.6、组合数

提供一组测试数据：

$(\frac{132}{66}) = 377'389'666'165'540'953'244'592'352'291'892'721'700$ ，模数为 998244353 时为 $241'200'029$ ； $10^9 + 7$ 时为 598375978。

组合数封装

```

1  struct Comb {
2      int n;
3      vector<Z> _fac, _inv;
4
5      Comb() : _fac{1}, _inv{0} {}
6      Comb(int n) : Comb() {
7          init(n);
8      }
9
10     void init(int m) {
11         if (m ≤ n) return;
12         _fac.resize(m + 1);
13         _inv.resize(m + 1);
14         for (int i = n + 1; i ≤ m; i++) {
15             _fac[i] = _fac[i - 1] * i;
16         }
17         _inv[m] = _fac[m].inv();
18         for (int i = m; i > n; i--) {
19             _inv[i - 1] = _inv[i] * i;
20         }
21         n = m;
22     }
23
24     Z fac(int x) {
25         if (x > n) init(x);
26         return _fac[x];
27     }
28
29     Z inv(int x) {
30         if (x > n) init(x);
31         return _inv[x];
32     }
33
34     Z c(int x, int y) {
35         if (x < 0 || y < 0 || x < y) return 0;
36         return fac(x) * inv(y) * inv(x - y);
37     }
38
39     Z P(int x, int y) {
40         if (x < 0 || y < 0 || x < y) return 0;
41         return fac(x) * inv(x - y);
42     }
43 };
44
45 comb(1 << 21);

```

取模组合数封装

基于自动取模类 Z 。

```

1  template <typename T = Z>
2  class Comb {
3  private:
4      int N;
5      std::vector<T> _Factor;
6      std::vector<T> _InvFactor;
7      std::vector<T> _Inv;
8  public:
9      Comb() : N{0}, _Factor{1}, _InvFactor{1}, _Inv{0} {}
10
11     explicit Comb(int n) : Comb() {
12         Init(n);
13     }
14
15     void Init(int M) {
16         if (M ≤ N) {
17             return ;
18         }
19
20         _Factor.resize(M + 1);
21         _InvFactor.resize(M + 1);
22         _Inv.resize(M + 1);
23
24         for (int i = N + 1; i ≤ M; i++) {
25             _Factor[i] = _Factor[i - 1] * i;
26         }
27
28         _InvFactor[M] = _Factor[M].inv();
29         for (int i = M; i > N; i--) {
30             _InvFactor[i - 1] = _InvFactor[i] * i;
31             _Inv[i] = _InvFactor[i] * _Factor[i - 1];
32         }
33
34         N = M;
35     }
36
37     Z Fac(int M) {
38         if (M < 0) {
39             return 0;
40         }
41         if (M > N) {
42             Init(std::max(M, N * 2));
43         }
44         return _Factor[M];
45     }
46
47     Z InvFac(int M) {

```

```

48     if (M < 0) {
49         return 0;
50     }
51     if (M > N) {
52         Init(std::max(M, N * 2));
53     }
54     return _InvFactor[M];
55 }
56
57 Z Inv(int M) {
58     if (M == 0) {
59         return 0;
60     }
61     if (M > N) {
62         Init(std::max(M, N * 2));
63     }
64     return _Inv[M];
65 }
66
67 Z C(int N, int M) {
68     if (N < M or M < 0) {
69         return 0;
70     }
71     return Fac(N) * InvFac(M) * InvFac(N - M);
72 }
73
74 Z A(int n, int m) {
75     if (n < m or m < 0) {
76         return 0;
77     }
78     return Fac(n) * InvFac(n - m);
79 }
80
81 Z Stirling2(int n, int k) {
82     if (n < k or k < 0) {
83         return 0;
84     }
85     if (n == k or k == 1) {
86         return 1;
87     }
88     if (k == 0) {
89         return (n == 0) ? 1 : 0;
90     }
91
92     Z res = 0;
93     for (int i = 0; i ≤ k; i++) {
94         Z term = C(k, i) * Z(i).pow(n);
95         if ((k - i) & 1) {
96             res -= term;
97         } else {
98             res += term;
99         }

```

```
100     }
101     return res * InvFac(k);
102 }
103
104 Z Catalan(int n) {
105     if (n < 0) return 0;
106     return C(2 * n, n) * Inv(n + 1);
107 }
108 };
109
110 constexpr int MAXN = 5e5 + 25;
111 Comb◇ comb(MAXN);
```